# CGNS Proposal for Extension 0049

## CGNS Compatibility Version Control

Version 1.3

August 4, 2026

| | |
|---|---|
| **CPEX#** | 0049 |
| **Scope** | Compatibility version control for improved interoperability |
| **Champion** | M. Scot Breitenfeld |
| **Organization** | The HDF Group |
| **E-mail** | brtnfld@hdfgroup.org |
| **GitHub Issue** | `https://github.com/CGNS/CGNS/issues/915` |
| **Reference Impl.** | `https://github.com/brtnfld/CGNS/tree/915` |
| **Target Release** | CGNS v5.0.0 |
| **Date First Posted** | March 28, 2026 |
| **Date of Last Revision** | August 4, 2026 |
| **SIDS Status** | accepted; awaiting implementation (adopted 2026-08-04) |
| **Filemap Status** | accepted; awaiting implementation (adopted 2026-08-04) |
| **MLL Status** | accepted; awaiting implementation (adopted 2026-08-04) |

# Contents

# 1 Abstract

This CPEX proposes a compatibility version control mechanism for the CGNS Mid-Level Library (MLL), inspired by the HDF5 library's `H5Pset_libver_bounds` facility. The current CGNS library rejects a file whose recorded `CGNSLibraryVersion_t` carries a higher *major* version than the running library, and warns but proceeds when the excess is confined to the lower-order digits—in neither case asking whether the file actually contains version-specific features that would prevent correct interpretation. The rejection unnecessarily breaks interoperability between tools compiled against different CGNS library releases; the warning path, conversely, admits files whose newer content the reader then silently ignores.

The extension introduces API functions and configuration options that allow applications to constrain the version they *write*, and shifts read-time version enforcement from a simple numeric comparison to a feature-aware compatibility check. An accompanying *automatic write-version* mode selects the minimum necessary version based on the features actually written, maximizing compatibility without requiring applications to hard-code version numbers.

Critically, the extension **redefines** `CGNSLibraryVersion_t` to record the minimum library version required to read the file, rather than the version of the library that wrote it, and introduces a new `CGNSWritingLibraryVersion_t` node to carry the provenance information that the original node used to hold. This redefinition is what allows *already-deployed* CGNS libraries—which cannot be modified—to accept newer files without change, and is therefore essential to the interoperability goal rather than incidental to it.

# 2 Motivation and Rationale

## 2.1 Problem Statement

When opening a CGNS file, the library currently enforces the following logic in `cg_open`:

```c
if (cg->version > CGNSLibVersion) {
    /* Reject if major version of file > major version of library */
    if ((cg->version / 1000) > (CGNSLibVersion / 1000)) {
        cgi_error("A more recent version of the CGNS library "
                  "created the file. Therefore, the CGNS library "
                  "needs updating before reading the file '%s'.",
                  filename);
        return CG_ERROR;
    }
    /* Warn only if different in second digit */
    if ((cg->version / 100) > (CGNSLibVersion / 100)) {
        cgi_warning("The file being read is more recent "
                    "that the CGNS library used");
```

```
    }
}
```

This version-number-only check creates several practical problems:

1. **False rejections.** A file written by CGNS 5.0 that uses only features present since CGNS 4.x will be rejected by a CGNS 4.x library, even though it could be read correctly.
2. **Blocked tool chains.** Widely-used post-processing tools (Tecplot, ParaView, VisIt) may ship with an older embedded CGNS library. Users who generate files with a newer library version cannot visualize their own data without recompiling or downgrading.
3. **Hindrance to incremental adoption.** Organizations running mixed CGNS library versions across production codes, mesh generators, and post-processors are forced into synchronized upgrades—an unrealistic constraint in large multi-team environments.

## 2.2 Why the Remedy Must Be in the File, Not the Reader

The library that performs the rejection is the *old* one. This observation constrains the design far more tightly than it may first appear, and it is the reason this proposal redefines an established node rather than adding a purely additive one.

Consider the concrete case in problem 2 above: a CGNS 4.4 library embedded in a commercial post-processor refuses a file written by CGNS 5.0. Any read-side logic introduced by this CPEX—a feature registry, a compatibility check, a new metadata node—resides in libraries built *after* the CPEX is adopted. The embedded 4.4 binary contains none of it. It reads `CGNSLibraryVersion_t`, observes 5.0, and rejects. A new node placed alongside that field is, by design, silently ignored by exactly the readers whose behavior needs to change.

It follows that:

- A purely additive file-format change (a new node carrying the minimum required version, leaving `CGNSLibraryVersion_t` untouched) delivers **no benefit to any library already in the field**. Its first useful transition would be from the release following this CPEX to the one after that.
- The *only* value an unmodified older reader consults is `CGNSLibraryVersion_t`. Therefore the value written into that field is the entire compatibility lever available to this proposal.

Redefining `CGNSLibraryVersion_t` to mean "the minimum library version required to read this file" causes a CGNS 5.0 writer that uses only CGNS 3.1 features to stamp `3.10`. An unmodified, already-shipped CGNS 4.4 library accepts that file immediately—no backport, no recompilation, and no cooperation from the tool vendor. This is the mechanism that makes the interoperability goal reachable, and Section 3.2 adopts it.

The provenance information that `CGNSLibraryVersion_t` formerly carried is not discarded: it moves to a new `CGNSWritingLibraryVersion_t` node, which only CPEX-0049-aware readers need to interpret. This allocation is deliberate—older readers cannot be taught to look for a new node, whereas newer readers can.

### 2.3 Precedent: HDF5 Library Version Bounds

HDF5 solved an analogous problem with the `H5Pset_libver_bounds(fapl, low, high)` API, which lets applications declare the range of HDF5 library versions whose on-disk representations they are willing to produce or consume. This approach:

- Decouples the *library version* from the *SIDS version*.
- Allows older libraries to read newer files when no incompatible features are present.
- Gives applications explicit control over the compatibility vs. feature trade-off.

The present CPEX adapts this design for CGNS.

## 3 Detailed Description

### 3.1 Design Overview

The implementation introduces three accompanying changes to the CGNS Mid-Level Library:

1. **Version milestone constants** (`CG_LIBVER_*`) encoding recognized CGNS version milestones as plain integer macros.
2. **New API functions** for setting and querying compatibility version bounds (lower and upper) on both a global and per-file basis, plus a parameter-object API for thread-safe per-open configuration.
3. **Feature-aware open and write logic** in `cg_open` that replaces the current numeric-only version check with a feature-compatibility assessment, and an automatic write-version mode that records the minimum version required by the features actually written—derived at close from the file's own content (Section 3.6.1) rather than accumulated as it is written.

## 3.2 File Format: Redefined `CGNSLibraryVersion_t` plus New `CGNSWritingLibraryVersion_t` Node

This CPEX must record three pieces of information: (a) the minimum CGNS version required to read the file, (b) the version of the library that wrote it, and (c) a diagnostic bitmask indicating which version-specific features are present.

The adopted design **swaps which node carries which meaning**:

- `CGNSLibraryVersion_t` is **redefined** to record the minimum library version required to read the file. This is the only field an unmodified older reader consults (Section 2.2), so it must carry the value that governs acceptance.
- A new `CGNSWritingLibraryVersion_t` node records the version of the library that last wrote the file, preserving the provenance information formerly held by `CGNSLibraryVersion_t`.

**Terminology.** This document says "library version" where the SIDS says "version of the CGNS standard." The two are interchangeable *for this field* because CGNS numbers the standard and the reference library from one series, and because the quantity that governs acceptance is what a given library build can interpret. Where the wording matters normatively—the SIDS and file-mapping amendments of Section 3.11 and the box below—the standard's own term is used. A reader should not infer from "library version" that the field records anything about a particular implementation; that is what `CGNSWritingLibraryVersion_t` is for.

`CGNSWritingLibraryVersion_t`

```
CGNSWritingLibraryVersion_t := ``CGNSWritingLibraryVer-
sion''
Label = ``CGNSWritingLibraryVersion_t''
Data-Type = ``R4''
Dimension = 1, Dimension-Value = 1
Data = CGNSWritingLibraryVersion
```

The node is a child of the file root node, at the same level as `CGNSLibraryVersion_t` and `CGNSBase_t`. Its value is an `R4` real number using the same decimal convention as `CGNSLibraryVersion_t` (e.g., `5.00` for CGNS 5.0), so that a reader comparing the two values need implement only one encoding.

**Encoding (normative).** Readers recover the integer version from either node as $\lfloor v \times 1000 + 0.5 \rfloor$, and writers store $V/1000$ as a binary32 value. Stating the rule is necessary because values such as `3.1` are not exactly representable in binary32; applying it makes the round trip *exact*. For any integer $V$ in the range $[1000, 9999]$, the relative error of binary32 is at most $2^{-24}$, so the absolute error after the division and the multiplication is under $2 \times 9999 \times 2^{-24} < 0.0012$—three orders of magnitude below the 0.5 that rounding to nearest can absorb. No representable CGNS version can round-trip incorrectly under this rule.

`R4` is chosen over an `I4` node holding `3100` or `5000` directly for three reasons. The rule above eliminates the only technical objection to `R4`; the distribution already depends on it, since `cgnscheck` recovers the integer with exactly `(int)(1000.0 * v + 0.5)` and `cg_version` applies that same expression before normalizing further, so this CPEX documents existing practice rather than introducing a requirement. (`cg_version` then snaps the result to the nearest multiple of ten, `((v + 5) / 10) * 10`, and rejects anything outside $[1000, 99990]$. Both steps are harmless here and the snap is strictly a robustness gain: the encoding $\mathrm{MAJOR} \times 1000 + \mathrm{MINOR} \times 100 + \mathrm{PATCH} \times 10$ is a multiple of ten by construction, so no value this CPEX can record is altered by it. Implementations must not introduce a recorded minimum that is not a multiple of ten.) A reader holding the two version values side by side implements one decode rather than two. And the file mapping for the new node becomes textually identical to the existing one, which is what allows Section 3.11 to specify it as "mapped exactly as `CGNSLibraryVersion_t` is." An `I4` node would be marginally cleaner in isolation and worse in every relationship it participates in.

**Name.** `CGNSWritingLibraryVersion_t` is chosen over the shorter `CGNSWriterVersion_t` and `CGNSCreatorVersion_t`. "Creator" is factually wrong: the field records the last library to write the file, not the first. "Writer" is not a term of art anywhere in the SIDS, whereas "library" is the word the sibling node already uses, and CGNS labels favor explicit multi-word names over abbreviation (`ZoneGridConnectivity_t`, `ArbitraryGridMotion_t`). Because `CGNSLibraryVersion_t` no longer means what its name suggests, the companion's name has to carry the disambiguation on its own in a tree dump where the two appear as adjacent siblings; the longer name is the one that does. The diagnostic bitmask (`_CGNS_FeatureMask`, Section 3.8) is placed on `CGNSLibraryVersion_t`, alongside the value it describes.

A CPEX-0049-aware file therefore reads:

```
/CGNSLibraryVersion          = 3.10   /* minimum required to read    */
    @_CGNS_FeatureMask       = 0x...  /* which features are present  */
/CGNSWritingLibraryVersion   = 5.00   /* library that last wrote it  */
/Base                        = ...
```

- `CGNSLibraryVersion_t` receives the minimum version required to read the file (determined automatically or set explicitly by the application).
- `CGNSWritingLibraryVersion_t` is **always** written as the current library version.
- Old libraries accept the file whenever the minimum required version is within their range, and silently ignore the new provenance node (standard CGNS

convention for unknown nodes).

- For legacy files, which have no `CGNSWritingLibraryVersion_t`, the two meanings coincide: `CGNSLibraryVersion_t` records the writing library's version, which is a conservative (never under-stated) proxy for the minimum required version. No migration of existing files is needed.

**Key properties of this design:**

- **Already-deployed readers benefit immediately.** This is the decisive property, and no purely additive design can provide it (Section 2.2).
- Provenance is preserved, in `CGNSWritingLibraryVersion_t`. The question "which library wrote this?" retains a definitive answer for all CPEX-0049-aware files.
- The major-version safety guard in Section 3.5 operates correctly on the value it already reads, with no reordering of the open path and no second node to consult before the file structure is parsed.
- Requires a SIDS addition (new node type), a formal *redefinition* of `CGNSLibraryVersion_t`, and a corresponding file-mapping update.
- Introduces one additional small node per file.

### 3.2.1 The Two Values Are Distinct In Memory

The redefinition applies to the *on-disk field*, not to the library's internal notion of a file's version. Confusing the two would be a serious implementation error, and the distinction is therefore normative.

A CGNS library consults a file's version for two unrelated purposes:

**Acceptance.** "May I open this file at all?" This needs the *minimum required* version, and it is the question an older reader is answering when it rejects a newer file. It is asked once, at open time.

**Interpretation.** "How were these bytes written?" This needs the *writing library's* version, because it selects among format generations—which element numbering convention applies, whether a dimension field has two entries or three, whether an unrecognized enumeration value should be tolerated as originating from a newer release. It is asked at dozens of points throughout the read path.

In the reference library, roughly fifty locations consult the version, and *every one of them is asking the interpretation question.* Some are exact-equality tests on a format generation, such as the CGNS 1.05 multiblock layout; others are thresholds, such as the CGNS 3.1 element renumbering, which *decrements* element type identifiers when the file predates `3100`.

It follows that the library's in-memory version field must continue to hold the **writing library's version**:

- The internal version field is populated from `CGNSWritingLibraryVersion_t` when present, and from `CGNSLibraryVersion_t` otherwise—which for legacy files *is* the writing library's version. Every existing interpretation site is therefore correct without modification.
- The value of `CGNSLibraryVersion_t` is retained separately and used only for the acceptance decision in Section 3.5 and for the `min_version` output of `cg_get_libver_bounds`.
- `cg_version()` is **unchanged in both signature and meaning**: it continues to report the version of the library that wrote the file. No existing caller is affected, and no new provenance accessor is required.

This separation is what makes the redefinition tractable. Without it, an AUTO-written file declaring `1.05` would drive a modern HDF5 file down the CGNS 1.05 parsing path, and the twenty-four graceful-degradation sites—which ask whether a file is newer than the running library—would turn warnings into hard errors for exactly the forward-compatible files this CPEX exists to admit. With it, no interpretation site changes at all.

**Consequences that must be documented.** Two costs follow directly from the redefinition and should not be understated:

1. **The node name no longer describes its contents.** A field named `CGNSLibraryVersion_t` that holds a *requirement* rather than a *library version* is a permanent ergonomic cost of this design. The name is retained because changing it would forfeit the entire benefit—older readers look up that exact name. The MLL reference manual and the SIDS must state the meaning prominently.
2. **An under-stated minimum can mislead an older reader.** If the recorded minimum is too low, an older library may accept a file containing features it cannot interpret. Because the value is derived at close from the file's own content (Section 3.6.1), this cannot be caused by a forgotten `cgi_require_version()` call—that mechanism is no longer the source of truth, and its omission has only the benign consequence described there. Exactly two causes remain: a feature with no detector in the registry, which the completeness test of Section 3.6 is mandated to catch, and content written out-of-band through low-level escapes, which the derivation cannot see at all (Section 3.7). Note the scope of the risk precisely: because interpretation within a CPEX-0049-aware library is driven by the provenance value (Section 3.2.1), such a file is still read correctly by any library that understands its features. The exposure is to *older* readers, and it is inherent in telling an older reader that a newer file is safe—the central premise of this CPEX rather than an incidental defect. Note also that an under-stated

minimum that falls below an *encoding generation* boundary is worse than one that merely omits a feature, because the older reader then mis-decodes data it would otherwise handle correctly; the AUTO floor rules in Section 3.6 exist to prevent this, and the completeness test mandated there is the control for both cases.

**On redefining an established node.** The redefinition is narrower than the node's name suggests, for two independent reasons. *First, the standard has already moved once.* The SIDS (*Hierarchical Structures*, "CGNS Version") states that "historically, the version number was used to describe the version of the Mid-Level Library used to write or modify the file. . . With the advent of new libraries that can read and write CGNS databases, the node is now defined as the version of the CGNS standard," and the file mapping defines the node as storing "the version number of the CGNS standard with which the file is consistent." The library-version reading is therefore already documented as *historical*; the current normative meaning is a statement about the file, not about the writer. *Second, the reference library does not implement even that.* Writing an ADF2-format file stamps `2.54` regardless of the writing library's version, and `CG_MODE_MODIFY` rewrites the field with the *modifying* library's version—so in practice the field means "the last library to write this file, clamped by container format." This CPEX moves the node from "the standard version this file is consistent with" to "the minimum standard version required to read this file"—a sharpening of an existing definition rather than a reversal of it.

### 3.2.2 Placement of `CGNSWritingLibraryVersion_t`

A secondary design question arises: should `CGNSWritingLibraryVersion_t` be placed as a *sibling* of `CGNSLibraryVersion_t` at the file root, as a *child* of `CGNSLibraryVersion_t`, or as a hidden *attribute* on that node?

#### Hidden attribute on `CGNSLibraryVersion_t`

Storing the writing version as an attribute (as `_CGNS_FeatureMask` is stored) would require no new SIDS node and no root-schema amendment, since attributes are not SIDS nodes. This option is *rejected*: provenance is read by humans during debugging, auditing, and compliance review, and a value invisible to generic tree-walking tools (`cgnsview`, `cg_array_info`, site scripts) is poor provenance. A diagnostic bitmask may reasonably be hidden; the answer to "which library wrote this file?" should not be.

#### Child of `CGNSLibraryVersion_t`

*Advantages:* all version metadata is co-located under one node, so readers needing both values traverse a single subtree; the root namespace does not grow; the SIDS amendment is scoped to an existing node type rather than the root-node schema.

*Disadvantages:* `CGNSLibraryVersion_t` is currently defined as a scalar leaf node, and promoting it to a container changes its character in the SIDS type system—a structural change on top of the semantic redefinition this proposal already requires of that node. Concentrating both changes on one node maximizes the risk that an existing validator rejects the file outright rather than merely warning.

**Root sibling (adopted)**

*Advantages:*

- `CGNSLibraryVersion_t` remains a scalar leaf node. Its *meaning* changes; its *structure* does not. Older readers parse it exactly as before.
- Clean separation of two independent facts: "requires at least version $X$" (`CGNSLibraryVersion_t`) and "last written by version $Y$" (`CGNSWritingLibraryVersion_t`).
- Consistent with all existing root-level CGNS metadata nodes, which are siblings rather than children of one another.
- Provenance is visible at the top level of any tree dump, where a reader looking for it will actually find it.

*Disadvantages:*

- Root namespace grows by one node type. The exposure here is smaller than it appears: the file mapping explicitly allows nodes below the root other than `CGNSBase_t`, and `cgi_read()` enumerates only `CGNSBase_t` at the root, so the MLL ignores the new node without modification. The residual risk is limited to validators that enumerate expected root children and report the rest, such as `cgnscheck` (see Section 3.10).
- Readers that need both version values must visit two sibling nodes rather than one parent.

Root sibling placement is adopted in this proposal.

**Naming.** The choice of `CGNSWritingLibraryVersion_t` over the shorter alternatives is argued in Section 3.2.

## 3.3 Version Constants: `CG_LIBVER_*`

Version milestone constants are defined in `cgnslib.h` as plain integer macros using the library's existing encoding—the dotted version multiplied by 1000, i.e. $\text{MAJOR} \times 1000 + \text{MINOR} \times 100 + \text{PATCH} \times 10$—which is the same encoding used by `CGNS_VERSION` and by the `CGNSLibraryVersion_t` node (`cgnslib.c`: "*Version of the CGNSLibrary*1000*"). The patch term is not vestigial: existing releases and existing read-path tests use it (`CGNS_COMPATVERSION` is 2540 for CGNS 2.54, `CG_LIBVER_EARLIEST` is 1050 for CGNS 1.05, and `cgnscheck` tests `FileVersion >= 3140`).

```c
/* Named constants are defined only for versions where a SIDS change
 * was introduced that a feature detector in the registry recognizes,
 * or that the encoding-generation floor computation refers to.
 * Intermediate releases with no SIDS changes (e.g., 3.3, 4.1-4.4)
 * are omitted; use the raw integer encoding for those if needed:
 *   MAJOR*1000 + MINOR*100 + PATCH*10   (e.g., 4400 for CGNS 4.4,
 *                                        1050 for CGNS 1.05)
 */
#define CG_LIBVER_EARLIEST  1050  /* CGNS 1.05 (oldest library version) */
#define CG_LIBVER_V12       1200  /* CGNS 1.2  - CGNSBase_t gained CellDimension
                                   *             and PhysicalDimension fields;
                                   *             referred to by the encoding floor */
#define CG_LIBVER_V30       3000  /* CGNS 3.0  - Extended element types */
#define CG_LIBVER_V31       3100  /* CGNS 3.1  - Reordered element types */
#define CG_LIBVER_V32       3200  /* CGNS 3.2  - NGON/NFACE v3.2 format */
#define CG_LIBVER_V40       4000  /* CGNS 4.0  - ElementStartOffset */
#define CG_LIBVER_V45       4500  /* CGNS 4.5  - Particles (CPEX 0046) */
#define CG_LIBVER_V50       5000  /* CGNS 5.0  - High-order elements (CPEX 0045) */
#define CG_LIBVER_LATEST    5000  /* Compile-time library version */
#define CG_LIBVER_AUTO        -1  /* Automatic write-version selection */
#define CG_LIBVER_DEFAULT      0  /* No ceiling: track running library */
```

`CG_LIBVER_EARLIEST` denotes the oldest CGNS library version. `CG_LIBVER_AUTO` is a sentinel directing the library to choose the minimum write version automatically (see Section 3.6).

`CG_LIBVER_LATEST` and `CG_LIBVER_DEFAULT` are distinct on purpose, and the distinction matters in an extension whose subject is mixed-version deployment. `CG_LIBVER_LATEST` expands to a *numeric constant fixed when the calling code is compiled.* An application built against CGNS 5.0 and later linked against a CGNS 6.0 shared library still passes `5000`, which the library cannot distinguish from a deliberate ceiling—so if `CG_LIBVER_LATEST` were also the default, upgrading the library under an unchanged application would silently forbid it from writing 6.0 features. `CG_LIBVER_DEFAULT` is not a version number and therefore cannot go stale: it means "no ceiling—use whatever the library linked at run time supports."

This is a deliberate improvement on the cited precedent rather than an oversight in it. HDF5's `H5F_LIBVER_LATEST` is a macro aliasing a specific enumerator (`#define H5F_LIBVER_LATEST H5F_LIBVER_V110` in the 1.10 series), so an HDF5 application passing it is pinned in exactly this way. Both spellings remain useful in CGNS: `CG_LIBVER_DEFAULT` for "keep up with the library" and `CG_LIBVER_LATEST` for "never emit anything newer than what I was built against," which is a legitimate request for reproducible output. The internal feature name and detector associated with each constant are catalogued in the registry table in Section 3.7.

## 3.4 New MLL Functions

### 3.4.1 Global Bounds via `cg_configure`

The global version bounds are set and queried through the existing `cg_configure` interface using the following new tokens:

```
#define CG_CONFIG_LIBVER_LOW         7  /* Set lower bound */
#define CG_CONFIG_LIBVER_HIGH        8  /* Set upper bound */
#define CG_CONFIG_GET_LIBVER_LOW     9  /* Query lower bound */
#define CG_CONFIG_GET_LIBVER_HIGH   10  /* Query upper bound */
```

`7`–`10` are the next four unassigned values. The library currently assigns `1`–`6` (`CG_CONFIG_ERROR` through `CG_CONFIG_RIND_INDEX`) at the MLL level, `201`–`210` for the HDF5/cgio options, `401` for `CG_CONFIG_GET_MAXIMUM_FILES`, and `1000` for `CG_CONFIG_RESET`. The binding constraint is the dispatch rule in `cg_configure`: any option greater than `100` is forwarded to `cgio_configure`, so an MLL-level token must be $\leq 100$.

**Defaults:** `low = CG_LIBVER_AUTO`, `high = CG_LIBVER_DEFAULT`.

```
/* Set lower bound (write-version floor) */
cg_configure(CG_CONFIG_LIBVER_LOW,
             (void *)(intptr_t)CG_LIBVER_AUTO);

/* Set upper bound (write ceiling; no effect on reads) */
cg_configure(CG_CONFIG_LIBVER_HIGH,
             (void *)(intptr_t)CG_LIBVER_V40);

/* Query current bounds */
int lo, hi;
cg_configure(CG_CONFIG_GET_LIBVER_LOW,  &lo);
cg_configure(CG_CONFIG_GET_LIBVER_HIGH, &hi);
```

**Scope:** Global—applies to all subsequent `cg_open` calls until changed. Not thread-safe; use the parameter-object API (Section 3.4.5) for thread-safe per-file configuration.

### 3.4.2 Library Version Bounds Functions

The following functions provide per-file control—including post-open adjustment and thread-safe multi-file scenarios:

```c
int cg_set_libver_bounds(int fn,
                               int low, int high);

int cg_get_libver_bounds(int fn,
                               int *low, int *high,
                               int *min_version);
```

`cg_set_libver_bounds` applies to the already-open file `fn` and is not retroactive: the acceptance decision of Section 3.5 was taken at open time and is not revisited, so a `high` set afterwards governs only subsequent write calls and the close-time stamp, and a `low` set afterwards raises the floor of the recorded minimum without ever lowering a value the file's contents already require. It returns `CG_ERROR` if `low` exceeds `high`, or if the minimum already required by data written so far exceeds the requested `high`.

`cg_get_libver_bounds` retrieves version information for an open file. Any output pointer may be `NULL` to skip that value:

`low` Current configured lower bound ($O(1)$). Pass `NULL` to skip.

`high` Current configured upper bound ($O(1)$). Pass `NULL` to skip.

`min_version` Minimum CGNS version required to read the file. **Only computed when non-`NULL`**. For a CPEX-0049 file open for reading this is the recorded value and costs $O(1)$; it incurs an $O(N_{\text{zones}} \times N_{\text{sections}})$ derivation only for a legacy file, or for any file open in a writable mode, where the value must be derived from the tree as it currently stands (Section 3.4.3). The result may be cached only in `CG_MODE_READ`, where the tree cannot change; in writable modes it must be re-derived on each call, since it has to reflect deletions as well as additions. Pass `NULL` to avoid the derivation entirely.

**Terminology note.** The three output parameters serve distinct purposes and should not be confused:

- `low` is the *configured lower bound*—the minimum SIDS version produced on write. When set to `CG_LIBVER_AUTO`, the library determines this automatically from the features written. It is a policy setting, not a file property.
- `high` is the *configured upper bound*—the newest feature level the application is willing to *write*. It is also a policy setting, and like `low` it has no effect on which files can be opened (Section 3.5).
- `min_version` is the *floor the file requires*—the oldest library version capable of reading the features actually present. It is derived from the file's content, not from any user setting.

This NULL-gated design keeps the API surface small while preserving full cost control:

applications that only need the configured bounds pass `NULL` for `min_version` and pay $O(1)$; applications that need the computed minimum version pay the scan cost exactly once.

### 3.4.3 Querying the Minimum Required Version

Because `min_version` is the only output of `cg_get_libver_bounds` that can cost more than $O(1)$, and because it is the output most applications actually want, it is also exposed under its own name:

```c
int cg_get_min_version(int fn, int *min_version);
```

This is a thin forwarder to `cg_get_libver_bounds(fn, NULL, NULL, min_version)` and adds no capability. It is specified rather than left to the reader because a single-purpose name is the difference between a cost a caller chooses and a cost a caller stumbles into: a `NULL` in the fourth argument slot of a four-argument query is easy to fill in reflexively, whereas nobody calls `cg_get_min_version` by accident. The combined function is retained for symmetry with `H5Pget_libver_bounds` and for callers that want both the policy and the requirement in one call.

`cg_get_min_version` is also the sanctioned way to answer "will my downstream tool be able to read this file?", which is why the bounds themselves do not gate reads (Section 3.5):

```c
int min_ver;
cg_get_min_version(fn, &min_ver);
if (min_ver > CG_LIBVER_V40)
    fprintf(stderr, "warning: file needs CGNS %d.%02d; "
                    "the certified toolchain has 4.0\n",
            min_ver / 1000, (min_ver % 1000) / 10);
```

**Behavior of `min_version` in write mode.** On a file open for `CG_MODE_WRITE` or `CG_MODE_MODIFY`, `min_version` runs the same derivation that `cg_close` will run (Section 3.6.1) against the tree as it currently stands, and returns what would be recorded if the file were closed at that moment. It therefore reflects deletions as well as additions, and it never returns `CG_LIBVER_AUTO`: `low` is a policy input to the derivation, not a possible result of it. Where `low` is an explicit constant, the value returned is at least `low`. Because the derivation is a traversal rather than a lookup, this query costs $O(N)$ in write mode even for a CPEX-0049 file; the $O(1)$ path applies to reading a recorded value, which a file being written does not yet have.

By contrast, HDF5's `H5Pget_libver_bounds` retrieves only the configured policy (`low`, `high`); HDF5 provides no equivalent of the `min_version` output. The ability to query the minimum version actually required by file content is a capability unique to this CGNS extension.

### 3.4.4 Interaction with `cg_version`

`cg_version` is **unchanged in signature and in meaning**: it continues to report the version of the library that wrote the file. Under the redefinition adopted in Section 3.2 that value is read from `CGNSWritingLibraryVersion_t` when present, and from `CGNSLibraryVersion_t` otherwise—which for legacy files is the same thing. Existing callers that use `cg_version` to report provenance require no change, and this CPEX adds no separate provenance accessor.

The recorded *minimum required* version is obtained from the `min_version` output of `cg_get_libver_bounds`. For a CPEX-0049 file this is the value stored in `CGNSLibraryVersion_t` and is returned in $O(1)$; for a legacy file it is computed by the feature scan described in Section 3.7. No new entry point is needed for either value.

### 3.4.5 Parameter Object API

For per-open configuration—including version bounds, file type, and compression—a parameter object type `cg_parameters_t` is provided. This is an opaque pointer; all operations go through the API.

**Thread-safety note.** The parameter object makes *configuration* per-call and race-free, but the MLL itself is not fully thread-safe: `cg_open` still accesses the global file table (`cg_files[]` array). Applications that open different files from concurrent threads must serialize their `cg_open`/`cg_close` calls or use external locking.

```
/* Opaque handle (NULL == use global configuration) */
#define CG_PARAMS_DEFAULT ((cg_parameters_t)NULL)

/* Parameter keys */
typedef enum {
    CG_PARAM_FILE_TYPE    = 100, /* CG_FILE_HDF5, CG_FILE_ADF, etc. */
    CG_PARAM_COMPRESS     = 101, /* Compression level: 0 (none) to 9 */
    CG_PARAM_LIBVER_LOW   = 102, /* Lower bound: CG_LIBVER_AUTO or
                                  * explicit CG_LIBVER_* constant */
    CG_PARAM_LIBVER_HIGH  = 103  /* Upper bound: CG_LIBVER_* ceiling */
} CG_PARAM_KEY;

/* Lifecycle */
int cg_params_create(cg_parameters_t *params);
int cg_params_destroy(cg_parameters_t  params);

/* Typed setters -- the documented interface */
int cg_params_set_libver_bounds(cg_parameters_t params,
                                int low, int high);
int cg_params_set_file_type(cg_parameters_t params, int file_type);
int cg_params_set_compress(cg_parameters_t params, int level);

/* Generic setter -- extension point for future keys */
int cg_params_set(cg_parameters_t params, int key, void *value);
```

**Typed setters are the documented interface.** The generic `cg_params_-`

`set(params, key, void *value)` is retained so that a future CPEX can add a key without adding a symbol, but it is not the interface applications are expected to use. A generic setter over `void *` defeats every compile-time check that matters here: it cannot tell `CG_PARAM_LIBVER_LOW` from `CG_PARAM_COMPRESS`, cannot tell a version constant from a compression level, and obliges every call site to write `(void *)(intptr_t)` around an integer. That cast is not cosmetic: it is the kind of construct that silently truncates on an LLP64 target if a caller writes `(void *)` alone. HDF5, the precedent this API follows in every other respect, uses typed per-property setters (`H5Pset_libver_bounds(fapl, low, high)`) for exactly this reason, and `cg_params_set_libver_bounds` mirrors it argument for argument. The generic form remains available for symmetry with `cg_configure`, whose established pattern this CPEX does not disturb.

Default values when a parameter object is created with `cg_params_create`:

- `CG_PARAM_LIBVER_LOW`: `CG_LIBVER_AUTO`
- `CG_PARAM_LIBVER_HIGH`: `CG_LIBVER_DEFAULT`
- `CG_PARAM_FILE_TYPE`: current `cgns_filetype` global
- `CG_PARAM_COMPRESS`: current `cgns_compress` global

A parameter object is passed to the 4-argument form of `cg_open_with_params`:

```c
int cg_open_with_params(const char      *filename,
                        int              mode,
                        cg_parameters_t  params,
                        int             *fn);
```

A parallel counterpart is required for the AUTO sequence of Section 3.6, which assumes each rank can carry a `low` setting through `cgp_open`:

```c
int cgp_open_with_params(const char      *filename,
                         int              mode,
                         cg_parameters_t  params,
                         int             *fn);
```

The Fortran bindings are named `cg_open_with_params_f` and `cgp_open_with_params_f` (Section 5), matching the C names exactly. Both fit comfortably within the 63-character identifier limit of Fortran 2003, so there is no reason to introduce a second vocabulary for the same function.

Passing `CG_PARAMS_DEFAULT` (a null handle) as the `params` argument is equivalent to calling the original three-argument `cg_open`: the global configuration applies.

**`cg_open` remains an ordinary function.** Earlier revisions of this proposal defined `cg_open` as a polymorphic macro that counted its arguments with `__VA_ARGS__` and dispatched to either the legacy three-argument function or `cg_open_with_params`. That device is withdrawn. Existing code is unaffected either way—`cg_open` keeps its signature and its behavior—so the macro's only benefit was letting new code spell the four-argument call `cg_open` instead of `cg_open_with_params`. Against that:

- **It would be the library's first variadic macro.** `__VA_ARGS__` appears nowhere in the CGNS sources today. Argument-counting macros are the fragile corner of C99 preprocessing: MSVC's traditional preprocessor requires an extra expansion indirection to make them work at all, and CGNS targets vendor compilers well outside the mainstream test matrix. Introducing that dependency on the library's single most-called entry point trades a naming convenience for a portability risk on every platform.
- **A macro is not a symbol.** Making `cg_open` a macro breaks every use that requires it to be a function: taking its address, wrapping it for tracing or profiling, resolving it through `dlsym`, and binding it from Python `ctypes`, Julia `ccall`, or any other FFI. These are ordinary uses in the CGNS ecosystem.
- **It needs an escape hatch, which is itself a tell.** The macro had to be suppressible with `CGNS_NO_MACROS`, so portable code would have had to be written to work both with and without it—leaving two supported spellings and testing neither well.

`cg_open_with_params` is therefore the sole four-argument entry point, `CGNS_NO_MACROS` is unnecessary and not defined, and the C API contains no construct a preprocessor can mishandle. This is also the stronger backward-compatibility claim: `cg_open` is not redefined at all.

**Design rationale: parameter object vs. extra arguments.** Four API patterns were considered for passing per-open configuration to `cg_open`:

1. **Extra positional arguments.**

   ```
   /* Hypothetical: direct extra args (low, high) */
   cg_open(file, mode, &fn,
           CG_LIBVER_V40, CG_LIBVER_LATEST);
   ```

   Simple for a fixed parameter set, but every new configuration option requires a new function signature or a new overloaded variant, and C has no way to add one without inventing a new name each time. Two options would exhaust the approach.

2. **Options struct with named fields.**

```c
/* Hypothetical: named-field struct */
cg_open_opts opts = {
    .low       = CG_LIBVER_V40,
    .high      = CG_LIBVER_LATEST,
    .file_type = CG_FILE_HDF5,
};
cg_open_ex(file, mode, &opts, &fn);
```

More discoverable—fields are visible in the struct definition—but not ABI-stable when the struct is passed by value. Adding a new field changes the struct layout for all callers, requiring either a versioned struct (with a `size` sentinel as used in POSIX's `struct addrinfo`) or a full API break. Accepting a *pointer* to the struct (rather than a copy) mitigates this: the function signature never changes when fields are added, and C99 designated initializers (`.field = value`) provide zero-default safety for new members. This pattern is common in modern C APIs (e.g., Vulkan's `VkInstanceCreateInfo`). The trade-off is that the struct definition must remain in a public header, exposing implementation details that an opaque handle hides.

3. **Variadic key–value pairs.**

```c
/* Hypothetical: key-value varargs (as used by cg_goto) */
cg_open_ex(file, mode, &fn,
           CG_PARAM_LIBVER_LOW,  CG_LIBVER_V40,
           CG_PARAM_LIBVER_HIGH, CG_LIBVER_LATEST,
           CG_PARAM_END);
```

CGNS already uses this pattern in `cg_goto`, which takes variadic `(label, index)` pairs terminated by a sentinel. The approach is familiar within the library, requires no heap allocation, and avoids a separate create/destroy lifecycle. However, it is not type-safe—all values must be passed as `int` or cast through a common type—and it cannot be passed as a single opaque handle to a downstream function without reconstructing the argument list. It also cannot hold non-integer parameters (e.g., future string-valued options) without a type-tag convention.

4. **Property object / opaque handle (chosen design).**

```c
cg_params_create(&params);
cg_params_set_libver_bounds(params, CG_LIBVER_V40,
                                    CG_LIBVER_DEFAULT);
cg_open_with_params(file, mode, params, &fn);
cg_params_destroy(params);
```

More verbose for simple cases, but ABI-stable: new parameters are added as new typed setters, or as new `CG_PARAM_*` keys on the generic setter, with no change to any existing function signature or struct layout. It also directly mirrors HDF5's

File Access Property List pattern (H5Pcreate/H5Pset_*/H5Fopen), which is already familiar to the scientific computing community—including its use of typed per-property setters, adopted above.

The property object was chosen because CGNS library releases are infrequent, ABI compatibility across releases is important to users, and the original three-argument cg_open is left untouched, so existing code requires no changes.

**Both entry points are exported symbols.** The C API consists of two ordinary functions:

```
/* Existing API: strict 3-argument function, unchanged */
CGNSDLL int cg_open(const char *filename, int mode, int *fn);

/* Explicit 4-argument API for per-open configuration */
CGNSDLL int cg_open_with_params(const char *filename, int mode,
                                cg_parameters_t params, int *fn);
```

Both symbols are exported from the shared library, and language bindings that use dlsym or FFI (Python ctypes, Java JNI, Julia ccall) resolve either one directly. No preprocessor construct stands between the caller and the symbol.

### 3.5 Modified cg_open Behavior

When a file is opened for reading (CG_MODE_READ or CG_MODE_MODIFY), the revised cg_open performs the following checks in order:

1. **Read version metadata.** Read the CGNSLibraryVersion_t node, which is present in every file the MLL has written. (It is not strictly guaranteed: a file lacking the node is treated by cg_version as CGNS 3.2. Under the redefinition that assumption reads as "requires 3.2," which is conservative and changes nothing; the CPEX text must state it so the two readings do not diverge.) Under the redefinition adopted in Section 3.2, its value *is* the *effective file version* $V_{\text{file}}$—the minimum library version required to read the file—for both CPEX-0049 files and legacy files:

   - CPEX-0049 file: the writer recorded the minimum required version directly, so $V_{\text{file}}$ is exact.
   - Legacy file: the value is the writing library's version, which is a conservative over-estimate of the true requirement. This reproduces the current library's behavior exactly, and the optional feature scan in Step 4 can refine it.

   Because a single node carries the value in both cases, no branch on file provenance is required and no additional node need be read before the safety check below.

`CGNSWritingLibraryVersion_t` is read when present, and `CGNSLibraryVersion_t` is used in its place when it is absent. This yields $V_{\mathrm{writer}}$, which becomes the library's in-memory version field and governs *all* subsequent interpretation of the file's contents—but takes no part in the acceptance decision below. The separation is normative; see Section 3.2.1.

2. **Major version safety check.** If the major version of $V_{\mathrm{file}}$ exceeds the major version of the running library ($\lfloor V_{\mathrm{file}}/1000 \rfloor > \lfloor V_{\mathrm{lib}}/1000 \rfloor$), return `CG_ERROR` immediately. This is a *crash-prevention guard*, not a compatibility check: a major version increment may indicate fundamentally different internal data layouts that would corrupt memory if the library attempted to parse them. It must therefore run *before* Step 3 reads the full file structure. It is distinct from the feature compatibility check in Step 4, which operates on already-parsed in-memory structures and cannot substitute for this guard.

   Note that the guard is now applied to the minimum *required* version rather than the writing library's version. A file written by a CGNS 6.0 library that requires only CGNS 5.0 features declares `5.00` and is admitted by a CGNS 5.x library, which is the intended outcome. This also means the guard is only as trustworthy as the writer's declaration—see the second consequence listed in Section 3.2.1.

3. **Read file structure** via `cgi_read()`.

4. **Check feature compatibility.** If $V_{\mathrm{file}} > V_{\mathrm{lib}}$, the verdict depends on whether $V_{\mathrm{file}}$ is a requirement or an estimate, which Step 1 has already established:

   - **CPEX-0049 file.** The recorded value *is* the requirement. Return `CG_ERROR` on the value alone, in $O(1)$, with no inspection of the file's contents. Where `_CGNS_FeatureMask` is present it supplies the feature name for the diagnostic; it never changes the verdict (Section 3.8.1).

   - **Legacy file.** The value is a writing library's version and therefore an overestimate. Check the file's metadata for features introduced after the running library's version, and return `CG_ERROR` only if one is found, with a diagnostic naming the feature and the required minimum library version.

   That the recorded minimum is normative for the first case, rather than a hypothesis the reader may test, is argued below.

5. **Accept.** If no incompatibility is found, open the file successfully.

6. **Warn if appropriate.** If $V_{\mathrm{file}} > V_{\mathrm{lib}}$ but the file was accepted, issue a `cgi_warning` so that the application can log the condition.

7. **Interpretation uses provenance, not the acceptance value.** Once the file is accepted, every subsequent decision about how its bytes are encoded—and

every graceful-degradation path that tolerates an unrecognized enumeration value on the grounds that the file appears newer—uses $V_{\text{writer}}$, exactly as the current library uses its single version field. See Section 3.2.1. No existing interpretation site requires modification.

**Both bounds govern writing only.** Neither `low` nor `high` takes any part in the acceptance decision above. Whether a file can be opened is determined by what the running library can interpret—the major-version guard and the feature check—and not by a policy the application set for its own output. Three reasons make this the right rule rather than merely the simple one:

- **It is what the cited precedent does.** HDF5 specifies that `H5Pset_libver_bounds` "indicates which versions of the file format the library should use *when creating objects*," that the bounds are "imposed with each HDF5 library API call that creates objects in the file," and explicitly that "setting the LOW and HIGH values will not affect reading/writing existing objects, only the creation of new objects." Adopting HDF5's names for a materially different behavior would be a trap for precisely the audience most likely to recognize them.
- **The party the ceiling protects never sees it.** An application that sets `high` is running the new library and can read everything. The party that cannot cope is a downstream tool in another process, which never observes the setting. A ceiling is therefore meaningful as a constraint on what this application *emits* and meaningless as a constraint on what it *accepts*.
- **It removes a self-inflicted false rejection.** Under read-side enforcement, a solver that caps its output at CGNS 4.0 for a certified post-processor would thereby refuse to read a 4.5 restart file it is perfectly capable of reading.

An application that wants to know whether a file it has written will be readable by an older tool asks directly, with `cg_get_min_version` (Section 3.4.3); it does not need a read-side gate to find out. Because CGNS maintains full backward compatibility, older files are always readable regardless of either setting.

**The recorded minimum is normative; a reader does not second-guess it.** For a CPEX-0049 file, $V_{\text{file}}$ is what the writer declares to be required, and a conforming reader treats it as the answer rather than as a hypothesis to test against the file's contents. The distinction becomes observable only when a writer set an explicit `low` above what its content requires (Section 3.6): the declared value then exceeds the derived minimum, and a post-CPEX-0049 reader older than `low` is refused a file it might in fact have parsed. That outcome is intended, for three reasons.

- **An explicit `low` is a request, not an accident.** The default is `CG_LIBVER_AUTO`, under which declared and derived minima coincide and the case cannot arise. A writer that raises the floor has asked for the file to be marked as requiring that version; honoring the request is not a defect in the reader.

- **The mask cannot support the alternative.** Refining downward would mean reconstructing the derived minimum from the file, and `_CGNS_FeatureMask` does not carry enough to do it: the encoding-generation floors of Section 3.6 are not features and have no mask bits, so a file of plain `HEXA_8` sections carries an empty mask and a derived minimum of `CG_LIBVER_V31`. A reader that accepted on the grounds that every set bit was one it recognized would mis-decode precisely the data those floors exist to protect.
- **It is the only remedy for out-of-band writes.** Section 3.7 directs users who write SIDS nodes through low-level escapes to set the floor explicitly, because the derivation cannot see that content. The mask cannot see it either, so a reader that overrode the declaration on the mask's authority would defeat the one protection such files have.

The alternative—recording the derived minimum separately so that a reader could accept when $V_{\text{derived}} \le V_{\text{lib}} < V_{\text{file}}$—was considered and rejected. It introduces a second on-disk value that must be kept in agreement with the first, which is the failure mode Section 3.8.4 exists to prevent; it has no realization in the ADF container; and it buys only the case a writer deliberately asked to forbid.

**Performance note.** For a file written by a CPEX-0049-aware library the acceptance decision is resolved entirely in $O(1)$ from the `CGNSLibraryVersion_t` value read in Step 1, optionally enriched for diagnostic purposes by `_CGNS_FeatureMask` (see Sections 3.2 and 3.8). *No CPEX-0049 file is ever scanned.* The feature scan, whose cost is linear in the total number of element sections in the file, survives only on the legacy branch of Step 4—and that branch is unreachable for conforming libraries, by the following invariant.

**Adoption invariant (normative).** CPEX-0049 is adopted from a definite release onwards: every library whose version is at least $V_{\text{cpex49}}$ implements it, and no earlier library does. A legacy file is by definition written by a library that does not, so $V_{\text{file}} = V_{\text{writer}} < V_{\text{cpex49}} \le V_{\text{lib}}$ for every CPEX-0049-aware reader. The trigger condition $V_{\text{file}} > V_{\text{lib}}$ cannot arise for a legacy file, so *every* open costs $O(1)$, legacy or not. Stating the invariant is what licenses that claim; a backport that gave the extension to some releases at a given version number but not others would void it.

The scan is nevertheless retained, for three reasons. A *non-conforming* writer—a fork, a vendor build, or a development snapshot numbered at or above $V_{\text{cpex49}}$ but built before the change landed—can emit a file that declares more than a conforming reader of the same number; a version node can be edited outside the MLL; and the traversal is in any case the same one that the close-time derivation (Section 3.6.1) and `cg_get_min_version` (Section 3.4.3) require, so retaining it costs no new code. It is a defensive path, not part of the specified open sequence for any conforming file.

## 3.6 Write-Mode Version Selection

The write version is the minimum required version recorded in the file's `CGNSLibraryVersion_t` node. It is governed by the lower bound (`low`). Two modes are supported:

### 3.6.1 The Recorded Minimum Is Derived at Close, Not Accumulated

The recorded minimum is **computed at `cg_close` time by running the feature detectors of Section 3.7 over the in-memory tree**, not accumulated during writing. This is normative, and it is the single most important implementation decision in this proposal after the redefinition itself.

The alternative—incrementing an effective version each time a write path announces a versioned feature through an internal `cgi_require_version()` call—was specified in earlier revisions and is withdrawn as the source of truth. It fails in a specific and unacceptable way: it makes the correctness of every file depend on a maintainer remembering to add a call. A future CPEX author who implements a new node and forgets one line produces files whose recorded minimum is too low, and an older library then accepts a file it cannot correctly interpret. That is a silent, data-corrupting failure introduced by an omission, in a mechanism whose entire purpose is to tell older readers that a file is safe.

Deriving at close inverts the failure mode. The detectors already exist because the read path needs them, so there is one source of truth rather than two mechanisms that must be kept in agreement; a missing detector is a missing *feature registration*, which the completeness test below catches directly. The derivation also:

- makes deletion handling correct in both directions. A minimum derived from what is present cannot over-state after a node is removed, which resolves Section 8.2 without decremental bookkeeping;
- eliminates the parallel reduction. Because CGNS requires metadata calls to be collective, every rank holds identical metadata at close and derives an identical answer with no communication;
- removes the write-path/read-path asymmetry that forced version constants such as `CG_LIBVER_V12` to exist with no corresponding detector.

The cost is one traversal of already in-memory metadata at close, linear in the number of element sections—negligible beside the I/O required to write those sections in the first place.

**`cgi_require_version()` is retained, with a different job.** The derivation happens at close, but a violation of the `high` ceiling must be reported *when the offending call is made*, not hours later when a long-running solver finally closes its file. HDF5 draws the same line: its bounds are “imposed with each HDF5 library API call that creates objects in the file," and violating calls fail. `cgi_require_version()` therefore remains at the write sites purely as this early-error mechanism. The safety property is that the two mechanisms now fail in opposite directions: a missing detector is caught by the completeness test, while a missing `cgi_require_version()` call merely postpones a `high` violation from the write call to the close call. *Neither omission can produce a file with an under-stated minimum.* The fragile mechanism has been reassigned to the benign failure.

### 3.6.2 Modes

`low = CG_LIBVER_AUTO` **(default).** The recorded minimum is the value derived at close (Section 3.6.1), raised to the floors below. For existing files (`CG_MODE_MODIFY`) the derivation runs over the modified in-memory tree, so the result reflects the file as it now stands rather than as it was opened. In both cases the file is stamped with the minimum version required by the features actually present, maximizing compatibility with older readers.

**Floor by container format.** `CG_LIBVER_EARLIEST` is not a valid floor for every file: a file in HDF5 format cannot be read by any library predating HDF5 support, irrespective of which SIDS features it uses, and an ADF2-format file is already clamped to `CGNS_COMPATVERSION`. The AUTO floor must therefore be the greater of `CG_LIBVER_EARLIEST` and the version that introduced the container format in use. Declaring a lower value would over-state compatibility in the one direction this CPEX must never over-state.

**Floor by encoding generation (normative).** A second and sharper floor is required, and it is not a feature floor. The read path uses the file's declared version to decide not only *whether* it understands the file but *how to decode bytes it already understands.* In `cgns_internals.c`:

- `cg->version < 3100`: every element type identifier in the open interval (`PYRA_5`, `NGON_n`) is remapped downward—`el_type--`, with `PYRA_14` $\to$ `PYRA_13`—to undo the CGNS 3.1 renumbering.
- `cg->version < 3000`: an element type identifier greater than `MIXED` is a hard error.
- `cg->version < 4000 && cg->version != 3400`: `NGON_n`/`NFACE_n` connectivity is rewritten from the pre-4.0 layout into `ElementStartOffset` form.
- `cg->version == 1050` (five read-path sites, two in `cgi_read_base` and three in `cgi_read_zone`, plus one write-side mapping in `cgi_write`): the pre-1.1 multiblock zone layout, including how `Zone_t` dimension values are interpreted.

An under-stated minimum therefore does something worse than admit an unreadable file: it instructs the reader to decode modern bytes with a legacy rule. A file containing ordinary `HEXA_8` sections that declares `3.00` has every element identifier decremented by a conforming reader—silent corruption rather than rejection, in precisely the already-deployed readers this CPEX invites in. This is not a hypothetical for AUTO: `HEXA_8` is not a new feature, so no feature detector fires, and a naive AUTO floor for a plain unstructured grid would be `CG_LIBVER_EARLIEST`.

Accordingly, the AUTO write version is normatively the maximum of (a) `CG_LIBVER_EARLIEST`, (b) the container-format floor, (c) the newest *encoding generation* the library's output assumes, and (d) any explicit `low`. Requirement (c) means the registry of Section 3.7 must carry, in addition to its feature detectors, an encoding floor for each node kind the library emits: any `Elements_t` section implies ≥ `CG_LIBVER_V31`, because identifiers are written in post-3.1 numbering; any `NGON_n`/`NFACE_n` section implies ≥ `CG_LIBVER_V40`, because connectivity is written in `ElementStartOffset` form; any `Zone_t` implies more than `1050`, because the 1.05 multiblock layout is not what is written. The practical effect is that the realistic AUTO floor for a modern file is CGNS 3.1–4.0 rather than CGNS 1.05. This costs the proposal nothing: its objective is that 4.x tools accept 5.x files, and a 3.1–4.0 floor delivers that while removing the misinterpretation hazard entirely.

**Completeness test (release blocker).** Deriving at close makes the recorded minimum only as good as the registry it is derived from: a feature with no detector is a feature that does not raise the minimum. A CI test must therefore write every known node type and element type to a fresh file in AUTO mode and assert that the resulting `CGNSLibraryVersion_t` equals the expected minimum. The same test must cover the encoding-generation floors above—in particular that a file whose features are all pre-3.1 but whose element identifiers are written in post-3.1 numbering is never stamped below `CG_LIBVER_V31`—since a missing encoding floor corrupts data in old readers rather than merely puzzling them. This test is the sole control on the correctness of the invitation this CPEX extends to older readers, and it should gate adoption.

`low` **= explicit version constant.** Setting `low` (via `CG_PARAM_LIBVER_LOW` or `CG_CONFIG_LIBVER_LOW`) to a specific `CG_LIBVER_*` constant establishes a *floor*: the recorded version is at least `low`, even if the features actually written would require less. A feature requiring *more* than `low` is not an error—the recorded version simply rises to meet it. Only a feature whose required version exceeds `high` returns `CG_ERROR`.

`CG_MODE_MODIFY` **handling.** When opening an existing file for modification, the two version nodes are treated differently:

- `CGNSWritingLibraryVersion_t` is **always** written as the current library version $V_{\mathrm{lib}}$, regardless of the `low` setting, and is created if absent. This is what makes the redefinition safe for provenance: every modification records who performed it.

- `CGNSLibraryVersion_t` is re-derived at close from the modified tree (Section 3.6.1). It is not carried forward from the existing value and is not constrained to increase: if the modification removed the only feature that required a high version, the recorded minimum correctly falls. Deriving from content rather than from history is what makes this safe—the value always describes the file as it is being written, never as some earlier version of it was.

**Existing behavior that must be removed.** `cg_open` today rewrites `CGNSLibraryVersion_t` with the running library's version on every `CG_MODE_MODIFY` open whose file value is lower (clamped to `CGNS_COMPATVERSION` for ADF2). Left in place, that block would overwrite the recorded minimum with the modifying library's version on the first modify, silently undoing the redefinition for every file it touches. It is replaced by the two rules above: the provenance node takes the value it used to write, and the minimum is re-derived.

**Parallel safety: no reduction is required.** The `CGNSLibraryVersion` node is a single shared object, so all ranks must write the same value to it. Deriving at close makes this automatic rather than something the library must arrange.

Parallel CGNS already requires metadata-defining calls (`cgp_zone_write`, `cgp_section_write`, and the rest) to be collective with identical arguments on every rank; only bulk data transfer is rank-local. Every rank therefore holds an identical in-memory metadata tree at close. Running a deterministic derivation over identical input yields identical output on every rank, with no communication, no ordering assumption, and no risk of a rank disagreeing.

This is worth stating explicitly because earlier revisions of this proposal specified an `MPI_Allreduce` with `MPI_BOR` over the feature mask at `cgp_close`. That reduction is now withdrawn. It was not merely redundant: `cgp_close` is presently a one-line wrapper around `cg_close` that performs no MPI collective of its own, so the reduction would have introduced a new synchronization point into the close path—and any new collective is a new opportunity for a deadlock when one rank takes an error path that the others do not. Deriving from replicated metadata avoids adding a collective at all.

An explicit `low` bound likewise applies identically on all ranks and needs no coordination. A `high` violation is detected where the offending write is issued and so is reported on that rank alone; as with any rank-local CGNS error, the application must treat it as a collective failure requiring an abort or coordinated recovery (Section 8.3).

### 3.7 Feature-Version Registry

The library maintains an internal, statically-compiled registry mapping CGNS features to the version in which they were introduced. Each entry has the form:

```
typedef struct {
    const char *feature_name;      /* e.g. "ParticleZone_t" */
    int         min_version;       /* CG_LIBVER_* constant  */
    int (*detector)(cgns_base *); /* returns 1 if feature present */
} cgns_feature_version;
```

The current registry (in `src/cgns_internals.c`):

| Feature | Min Version | Constant (§3.3) |
|---|---|---|
| `Extended_ElementTypes` | 3000 | `CG_LIBVER_V30` |
| `Reordered_ElementTypes` | 3100 | `CG_LIBVER_V31` |
| `NGON_NFACE_V32` | 3200 | `CG_LIBVER_V32` |
| `ElementStartOffset` | 4000 | `CG_LIBVER_V40` |
| `ParticleZone_t` | 4500 | `CG_LIBVER_V45` (CPEX 0046) |
| `ParticleCoordinates_t` | 4500 | `CG_LIBVER_V45` (CPEX 0046) |
| `ParticleSolution_t` | 4500 | `CG_LIBVER_V45` (CPEX 0046) |
| `ElementInterpolation_t` | 5000 | `CG_LIBVER_V50` (CPEX 0045) |
| `SolutionInterpolation_t` | 5000 | `CG_LIBVER_V50` (CPEX 0045) |
| `Unknown_Modern_Features` | `CG_LIBVER_LATEST` | Fail-safe |

**Note on `CG_LIBVER_V12`.** The constant is defined but has no registry entry. Earlier revisions, which bumped the version from the write path, used it when writing `CGNSBase_t` nodes, and the resulting asymmetry between a write-path version map and a read-path detector set had to be explained. Under the derivation rule of Section 3.6.1 there is only one mechanism and no asymmetry to explain: `CGNSBase_t` with `CellDimension` and `PhysicalDimension` is present in every modern file, so it contributes to the encoding-generation floor rather than to the feature registry, and a detector for it would return true unconditionally. The constant is retained because it names a real milestone that the floor computation refers to.

The fail-safe entry `Unknown_Modern_Features` catches any features not recognized by the library (currently, element type identifiers at or beyond `ElementType_MAX`, i.e. past `HEXA_125`), preventing silent data corruption when reading files written with future CGNS versions.

**Note: no `HighOrder_ElementTypes` entry.** Earlier revisions listed a third CGNS 5.0 entry under that name. It is removed because no detector can be written for it. CPEX 0045 as implemented adds `ElementInterpolation_t`, `SolutionInterpolation_t`, the `InterpolationType_t` enumeration and the `ElementBased` grid location, but *no new `ElementType_t` enumerants*: the enumeration ends at

`HEXA_125`, with `ElementType_MAX = 57` and an explicit "add new element types here (value 57+)" marker, on both the CPEX-0045 branch and the 5.0 development branch. A detector for the entry would therefore look for element types that either already exist in CGNS 3.x—and so must not require 5.0—or do not exist yet, in which case the fail-safe entry already covers them. The two remaining 5.0 entries are sufficient to require `CG_LIBVER_V50` for any file using CPEX-0045 features. Should a future revision of CPEX 0045 introduce element-type enumerants at 57 or above, they take the next free mask bit rather than reviving this one.

**Hard guard: modifying files that trigger the fail-safe.** When a file containing unknown element types is opened in `CG_MODE_MODIFY`, the library cannot safely *preserve* unknown data structures during a partial rewrite. For example, if a CGNS 5.0 library opens a CGNS 6.0 file containing `HEXA_216` elements and then modifies an unrelated zone, the write path has no knowledge of how to serialize the unknown element data back to disk. The result would be a corrupted or truncated file. Therefore, **`cg_open` must return `CG_ERROR`** when `Unknown_Modern_Features` is detected and the mode is `CG_MODE_MODIFY`. The error message must identify the unrecognized features so the user can decide whether to upgrade the library or open the file in `CG_MODE_READ` instead. Opening the same file in `CG_MODE_READ` remains permitted; only write-capable modes are rejected.

This guard operates on the legacy branch, where a scan runs. For a CPEX-0049 file it is subsumed: a registered feature the reader does not know raises the recorded minimum above the reader's own version, so the file is refused in $O(1)$ before any question of modifying it arises (Section 3.8.1). A CPEX-0049 file that reaches `CG_MODE_MODIFY` carrying unknown element types can only have been produced by a non-conforming writer or by an out-of-band write, and neither the scan nor the recorded minimum can protect against those.

During the feature-compatibility scan, each detector function examines the CGNS tree for the relevant node types. Detectors that operate on nodes not yet parsed into MLL data structures (e.g. `ElementInterpolation_t`) use `cgi_get_nodes()` to scan the raw file tree directly.

**Known limitation: low-level write bypasses.** The derivation of Section 3.6.1 runs over the MLL's in-memory tree. Nodes written through low-level escapes—direct `cgio_*` calls, or external HDF5/ADF tools operating on the file outside the MLL—never enter that tree and therefore cannot raise the recorded minimum. Such a file may declare a minimum lower than its content requires. Users who write SIDS nodes through low-level interfaces are responsible for setting the floor explicitly with `cg_configure(CG_CONFIG_LIBVER_LOW, ...)`, or for accepting that the resulting file may be misidentified as compatible by compatibility-checking readers. Note that deriving at close narrows this hole rather than widening it: content written through the MLL is covered whether or not any write path remembered to announce it, so only genuinely out-of-band writes escape.

## 3.8 Eliminating the $O(N)$ Feature Scan

The $O(N_{\text{zones}} \times N_{\text{sections}})$ feature scan is unacceptable for production HPC files, which routinely contain 100 000 or more unstructured blocks.

The redefined `CGNSLibraryVersion_t` node (Section 3.2) already eliminates this cost for the primary use case: the open-time compatibility check reads a single integer from the node and compares it against the running library's own version in $O(1)$. The configured bounds take no part in it (Section 3.5). **No feature scan is ever performed for a CPEX-0049 file.** The $O(N)$ scan exists only as a fallback for legacy files, which do carry `CGNSLibraryVersion_t`—every CGNS file does—but carry it with the older meaning and lack `CGNSWritingLibraryVersion_t`; and even for those it is unreachable in conforming deployments, by the adoption invariant of Section 3.5.

The `_CGNS_FeatureMask` attribute provides an additional **diagnostic** benefit: when a file is rejected, the bitmask identifies *which* features caused the incompatibility, enabling precise error messages (e.g., "file requires CGNS $\geq$ 4.5 for `ParticleZone_t`") without performing a full scan.

### 3.8.1 The Mask Is Diagnostic, Not Normative

The mask is specified as a **recommended, diagnostic** attribute. It is not required for correctness, its absence is not an error, and no reader is required to consult it. An earlier revision raised the opposite possibility—making it mandatory and requiring readers to reject any bit they do not recognize, as a general forward-compatibility contract. That reading is rejected, for a reason worth stating because it is the reason the redefinition earns its cost.

Consider what an unrecognized bit would actually mean. The reader has already accepted the file, which means the recorded minimum was within its range. An unknown bit therefore says: *a feature I do not recognize is present, and the writer declared that it does not require a newer library to read.* That is not a danger signal—it is the definition of a forward-compatible feature, and it is exactly the situation this CPEX exists to permit. Rejecting on it would refuse precisely the files the proposal was written to admit. Conversely, a feature that genuinely cannot be ignored raises the recorded minimum, and the older reader rejects the file in $O(1)$ on the version alone, having never looked at the mask.

**The scope of that argument is worth stating, because the mask is not symmetric.** It holds where $V_{\text{file}} \leq V_{\text{lib}}$: the file has been accepted, so the writer's declaration covers the reader, and an unknown bit is covered with it. Where $V_{\text{file}} > V_{\text{lib}}$ the declaration does *not* cover the reader, and the mask may not be used in the permissive direction either—not because an unknown bit is a hazard, but because a mask consisting entirely of *known* bits is still not evidence of readability: it omits the encoding-generation floors, which are not features (Section 3.5). In both directions the mask explains a verdict reached from the version; it never reaches one.

**The minimum-required version is the general fail-safe.** This is the property that a per-feature flag plane would otherwise have to supply, and one number supplies it for all future features at once, including node types and enumeration values that no detector in this registry anticipates. It is also why the registry's `Unknown_Modern_Features` entry is not redundant: it applies only to *legacy* files, whose declared version is a writer version rather than a requirement and therefore carries no such guarantee. For those files, scanning for unrecognized element types is a genuine, if partial, safety net; for CPEX-0049 files it is unnecessary.

The comparison with HDF5 is instructive here, because HDF5 does maintain exactly the flag plane just rejected. Its object-header messages carry `H5O_MSG_FLAG_FAIL_IF_UNKNOWN_ALWAYS` and `H5O_MSG_FLAG_FAIL_IF_UNKNOWN_AND_OPEN_FOR_WRITE`, and the library rejects unrecognized flag bits outright. HDF5 needs this because an object header has no field stating the minimum library version required to interpret the object: each message must carry its own verdict. CGNS, after this CPEX, has that field. The per-feature plane would be a second, weaker answer to a question the redefined node already answers exactly.

Two further consequences follow, and both are simplifications. The mask's absence in the ADF container—which has no attribute concept at all—is a non-issue rather than a specification gap: such files simply take the legacy path. And a mask lost in transit, for instance across a `cgio_copy_file`-based conversion, degrades diagnostics without ever affecting a correctness decision.

The reference implementation writes and reads this attribute for the HDF5 backend.

### 3.8.2 Specification

When a CPEX-0049-aware library closes a writable file, it writes a hidden attribute named `_CGNS_FeatureMask` on the `CGNSLibraryVersion_t` node. The attribute is stored as a **signed 64-bit integer** (`I8` in SIDS `DataType_t` terms). In the HDF5 backend this maps to `H5T_NATIVE_INT64`. Declaring the width in SIDS `DataType_t` terms rather than as a raw HDF5 type keeps the specification backend-neutral and gives tools that *do* inspect attributes (`cgnsview`'s attribute display, `h5dump`, site scripts using HDF5 directly) a well-defined type to read. It does not make the value visible to the node-oriented APIs: an attribute is not a CGNS node, so `cg_array_info` and `cgnscheck` cannot reach it—which is the same property that disqualified an attribute for *provenance* in Section 3.2.2 and is acceptable only because this value is diagnostic.

**Backend and API prerequisites.** Two prerequisites are not yet satisfied by the reference distribution and are in scope for the implementation. First, `cgio` exposes no attribute API (`cgns_io.h` has no attribute entry points), so writing and reading `_CGNS_FeatureMask` requires either new `cgio_*_attribute` calls or a backend-specific escape that bypasses the container abstraction; the former is preferred. Second, the ADF container has no attribute concept at all, so "an `I8` attribute" has no ADF realization: for ADF and ADF2 the mask must either be omitted—leaving those files on the legacy $O(N)$ path, which is harmless—or specified as a hidden child node. Relatedly, `cgio_copy_file` (used by `cgnsconvert`) copies nodes, not attributes, so the mask is dropped across a format conversion and must be recomputed by the converting tool (Section 3.10).

Although `I8` is signed, bit-level operations are well-defined in C for non-negative values, and the implementation should cast to `uint64_t` internally for bitwise manipulation. Each feature is assigned an explicit power-of-two mask constant, making bitwise operations natural (e.g., `if (mask & (CGNS_FEATURE_EXTENDED_ELEMENTS | CGNS_FEATURE_PARTICLE_ZONE))`):

```
/* The shift is performed in 64-bit: a plain (1 << n) is an int
 * expression and is undefined behavior for n >= 32, which would
 * silently break the first feature assigned to bit 32 or above. */
#define CGNS_FEATURE_BIT(n)                 (INT64_C(1) << (n))

#define CGNS_FEATURE_EXTENDED_ELEMENTS     CGNS_FEATURE_BIT( 0) /*   1 */
#define CGNS_FEATURE_REORDERED_ELEMENTS    CGNS_FEATURE_BIT( 1) /*   2 */
#define CGNS_FEATURE_NGON_V32              CGNS_FEATURE_BIT( 2) /*   4 */
#define CGNS_FEATURE_ELEMENT_START_OFFSET  CGNS_FEATURE_BIT( 3) /*   8 */
#define CGNS_FEATURE_PARTICLE_ZONE         CGNS_FEATURE_BIT( 4) /*  16 */
#define CGNS_FEATURE_PARTICLE_COORDS       CGNS_FEATURE_BIT( 5) /*  32 */
#define CGNS_FEATURE_PARTICLE_SOLUTION     CGNS_FEATURE_BIT( 6) /*  64 */
#define CGNS_FEATURE_ELEMENT_INTERP        CGNS_FEATURE_BIT( 7) /* 128 */
#define CGNS_FEATURE_SOLUTION_INTERP       CGNS_FEATURE_BIT( 8) /* 256 */
#define CGNS_FEATURE_UNKNOWN_MODERN        CGNS_FEATURE_BIT( 9) /* 512 */
/* Next free bit: 10 */
```

A set bit indicates the corresponding feature is present in the file. Mask assignments are permanent: once a bit position is assigned to a feature, it must never be reused, even if the feature is deprecated. Using explicit constants (rather than implicit array ordering) ensures that source-code refactoring of the registry table cannot silently change bit assignments and break backward compatibility.

Using a 64-bit type provides 64 feature slots. With the current registry at 10 entries and CGNS adding roughly one to two version-breaking features per major release, this provides ample runway without the overhead of a variable-length representation. If the registry ever approaches 64 entries, a follow-on CPEX can introduce `_CGNS_FeatureMask2` for the overflow bits, maintaining backward compatibility.

Future CPEXs that introduce version-breaking features must allocate the next available bit position and add the corresponding `CGNS_FEATURE_*` mask constant. Retired entries should be marked as reserved in the source.

#### 3.8.3 Open-time use

On `cg_open()`, compatibility checking proceeds as follows:

**The discriminator is the provenance node, and only that node.** A file is a CPEX-0049 file if `CGNSWritingLibraryVersion_t` is present, and legacy if it is absent (Section 3.11). The mask must not be used for this test: a CPEX-0049 file legitimately carries no mask when it lives in an ADF or ADF2 container, when it has passed through `cgio_copy_file`, or when a crash preceded `cg_close()` (Section 3.8.4). Treating those as legacy would read a recorded minimum as though it were a writing library's version.

1. **CPEX-0049 file** (`CGNSWritingLibraryVersion_t` present): `CGNSLibraryVersion_t` carries an exact requirement. Read it in $O(1)$ and compare it against the running library's version; the configured bounds take no part (Section 3.5). If the file is rejected and `_CGNS_FeatureMask` is present, use the bitmask to name the incompatible feature in the error message. **No tree walk is performed, in either outcome, and the mask never alters the outcome.**
2. **Legacy file** (`CGNSWritingLibraryVersion_t` absent): the `CGNSLibraryVersion_t` value is a writing library's version and therefore a conservative overestimate. If it alone permits acceptance, accept in $O(1)$; only if it would cause rejection is it worth refining, in which case fall back to the $O(N)$ tree walk. This preserves full backward compatibility with all pre-CPEX-0049 files.

**The legacy fallback is a defensive path, not a cost of normal operation.** By the adoption invariant of Section 3.5, a legacy file's declared version is below that of any CPEX-0049-aware reader, so the refinement in case 2 is unreachable in a conforming deployment: legacy files are accepted in $O(1)$ on the version alone. What remains reachable is a file from a non-conforming writer or one whose version node was edited outside the MLL—and for those the fallback is exactly the right behavior, since it grants such a file the same treatment the current library gives every file today.

When the walk does run, its cost is bounded in three ways. It walks in-memory data structures that `cgi_read()` has already parsed, so it issues no additional I/O. The short-circuit checker (`cgi_check_version_limit`) stops on the first incompatible feature, so the full $O(N)$ cost is incurred only when the file turns out to be *compatible* and every detector must confirm the absence of violations. And it is the same traversal the close-time derivation already performs (Section 3.6.1), so a file opened in a writable mode pays for it once regardless.

### 3.8.4 Maintenance

An `int64_t current_feature_mask` field is added to the internal `cgns_file` struct, holding the mask read at open so that a reader can produce a precise diagnostic without a tree walk. It is a cache of what the file said, not an accumulator of what the library has done.

The lifecycle is:

1. **Open (CPEX-0049 file):** read `_CGNS_FeatureMask` into `current_feature_mask` in $O(1)$. A CPEX-0049 file may legitimately carry no mask—an ADF or ADF2 container, a file that has passed through `cgio_copy_file`, a crash before `cg_close()`—in which case the field is left empty and the only loss is diagnostic precision. The acceptance decision is unaffected, having never consulted the mask.

2. **Open (legacy file):** no mask is present, and none can be. It is populated by the $O(N)$ tree walk if one is performed, and otherwise left empty.

3. **Write path:** nothing. The mask is not updated as features are written, because it is not the source of the recorded version.

4. **Close (writable modes):** the derivation of Section 3.6.1 produces the recorded minimum and the mask together, from the same traversal of the same tree, and both are written. Skipped entirely for `CG_MODE_READ`.

Deriving both values from one traversal is what guarantees they agree. An accumulated mask and a derived version—or two independently maintained values of any kind—could disagree, and a mask that contradicts the version it annotates is worse than no mask, since its only purpose is to explain that version.

**What "agree" means here.** The mask is an inventory of the features present in the tree, and the value it must agree with is the *derived* minimum computed from that same tree. The value *recorded* in `CGNSLibraryVersion_t` is the greater of the derived minimum and any explicit `low` (Section 3.6), so a file written with a raised floor carries a mask whose features imply less than the recorded value demands. That is not the contradiction guarded against here, for the same reason it is not a licence to refine on read (Section 3.5): the mask does not purport to state the requirement, and no reader consults it to override one. The contradiction that would matter is a mask that misdescribes the tree—a bit set for a feature that is absent, or clear for one that is present—and deriving both values in one pass is what makes that impossible.

**Crash and abort safety.** Because the feature mask is held in memory and only flushed at `cg_close()`, an application crash or `MPI_Abort()` mid-write will leave the file without an updated `_CGNS_FeatureMask`. This is by design: the file is likely truncated or inconsistent in that case, and no data corruption or false acceptance results from the mask itself, which carries no authority over any decision. Two sub-cases differ and should not be conflated. On a crash during `CG_MODE_WRITE` there is no mask and no recorded minimum, so the next reader has nothing to be misled by. On a crash during `CG_MODE_MODIFY` the mask and the recorded minimum both retain their pre-modification values and remain consistent with *each other* while describing the file as it was; if the crash landed after new content reached disk, the recorded minimum under-states it. That exposure belongs to the recorded minimum, not to the mask, it is inherent in any crash mid-rewrite, and it is bounded by the fact that such a file is already suspect. It is one more reason the mask is specified as diagnostic: a stale mask cannot turn a rejection into an acceptance.

**Legacy file healing.** When a legacy file—one with no `CGNSWritingLibraryVersion_t`—is opened in `CG_MODE_MODIFY` and closed, the close-time derivation of Section 3.6.1 runs as it does for any writable file, and the file acquires all three values: a `CGNSLibraryVersion_t` re-derived from its actual content, a `CGNSWritingLibraryVersion_t`, and a `_CGNS_FeatureMask`. No scan at *open* time is required to achieve this, and none is specified: the derivation at close already traverses the tree.

The benefit is not the elimination of a future scan—by the adoption invariant of Section 3.5, a CPEX-0049-aware reader was already accepting that file in $O(1)$. The benefit is that the re-derived value is typically *lower* than the writing library's version the file previously declared, so the file becomes readable by *older, unmodified* libraries that the original value shut out. A file stamped `4.40` because a CGNS 4.4 library wrote it, but containing only pre-3.1 features, is restamped `3.10` and is thereafter accepted by a CGNS 3.1 tool. That is the proposal's central benefit applied retroactively to existing data, and it follows from touching any node in the file—even a trivial coordinate update—without explicit user action.

## 3.9 SIDS Impact

This CPEX makes two changes to the SIDS, one semantic and one structural.

**SIDS amendments required.** **(1) Redefinition.** Two passages must be amended together: the SIDS *Hierarchical Structures* section ("CGNS Version"), which defines the node as "the version of the CGNS standard," and the file mapping's `CGNSLibraryVersion_t` node description, which defines it as "the version number of the CGNS standard with which the file is consistent." Both become "the minimum version of the CGNS standard required to read the file." The node's data type, dimension, and structure are unchanged; only its meaning changes. **(2) Addition.** The file mapping already permits additional root-level nodes—"a file may also contain other nodes below the root node beside `CGNSBase_t`, but these are not officially part of the CGNS database and will not be recognized by most CGNS software"—so `CGNSWritingLibraryVersion_t` is legal in an unamended file mapping today. The amendment is needed not to make the node *permissible* but to make it *official*: to give it a normative definition, place it in the same "exception" category as `CGNSLibraryVersion_t`, and oblige conforming software to preserve it. A corresponding file-mapping node description (Section 3.11) must be published with it.

The `_CGNS_FeatureMask` attribute must be documented in the SIDS file mapping as a hidden `I8` attribute on the `CGNSLibraryVersion_t` node.

Because the redefinition changes the meaning of a field that pre-CPEX-0049 files already contain, the SIDS text must also state the interpretation rule for legacy files explicitly: *when `CGNSWritingLibraryVersion_t` is absent, `CGNSLibraryVersion_t` carries the writing library's version, which is to be treated as a conservative upper estimate of the minimum requirement.* The presence or absence of the provenance node is what distinguishes the two readings, and it is the only signal a reader needs.

## 3.10 Tool Impact

The tools shipped with the reference distribution consume the affected field and require corresponding updates. These are part of the CPEX, not follow-on work:

- `cgnscheck` validates a file's structure against the SIDS rules in force at the file's declared version—some thirty `FileVersion`-keyed tests, including `FileVersion < 3200` guards on `NGON_n` usage—and compares that version against its own. Under the redefinition it must validate against $V_{\text{writer}}$, or a CGNS 5.0 file recording `3.10` would be checked against pre-3.2 rules and its own legitimate content reported as an error. The change is smaller than it appears: `cgnscheck` obtains the value from `cg_version` rather than by reading the node, so once `cg_version` returns provenance (Section 3.4.4) every one of those tests keys on $V_{\text{writer}}$ automatically. What remains is to accept `CGNSWritingLibraryVersion_t` as a legal root child rather than reporting it as unrecognized.
- `cgnsview` should display both values with their meanings, since the node name

alone is now misleading.

- `cgnsdiff` should not report a difference in `CGNSWritingLibraryVersion_t` as a data difference.
- `cgnsconvert` must carry both nodes across format conversions, and must recompute `CGNSLibraryVersion_t` when the target container format has a higher floor than the source (Section 3.6). Both nodes survive a conversion today, because `cgio_copy_file` copies the tree generically; the `_CGNS_FeatureMask` attribute does *not*, since that copy operates on nodes only, so the tool must recompute or re-emit the mask. An HDF5 → ADF2 conversion is the case where the container floor actually bites: the ADF2 target cannot honor a recorded minimum below `CGNS_COMPATVERSION`.

### 3.11 File Mapping Impact

One new node (`CGNSWritingLibraryVersion_t`) is added at the root level, mapped exactly as `CGNSLibraryVersion_t` is: an `R4` scalar. The `_CGNS_FeatureMask` attribute is placed on `CGNSLibraryVersion_t`. The mapping of `CGNSLibraryVersion_t` itself is unchanged—only its documented meaning.

Following the node-description prescription of the file mapping manual:

| **Node Attributes:** `CGNSWritingLibraryVersion_t` | |
|---|---|
| **Name** | `CGNSWritingLibraryVersion` |
| **Label** | `CGNSWritingLibraryVersion_t` |
| **DataType** | `R4` |
| **Dimension** | 1 |
| **Dimension Values** | 1 |
| **Data** | Version of the CGNS library that last wrote the file |
| **Children** | None |
| **Cardinality** | 0 or 1 (absent in pre-CPEX-0049 files) |
| **Parent** | Root node |

The cardinality is deliberately *0 or 1* rather than `1`: absence is meaningful, since it is the signal that `CGNSLibraryVersion_t` is to be read with its legacy meaning (Section 3.8). As with `CGNSLibraryVersion_t`, at most one such node may appear below the root, and its value applies to every `CGNSBase_t` in the file.

Implementations that do not recognize the new node or attribute must silently ignore them.

## 4 Backward Compatibility

The CPEX process states that "no additions or changes to the CGNS standard will be adopted—without overwhelmingly compelling reasons—which invalidate existing software or data." Because this proposal redefines an established node rather than only adding one, it is measured against that requirement directly. It invalidates no existing data: every pre-CPEX-0049 file remains valid and yields the same accept/reject outcome as today. It invalidates no existing software either, with one bounded exception—tools that read `CGNSLibraryVersion_t` directly and *report* it as provenance will report a different number for new files. Those tools are enumerated in Section 3.10 and their updates are in scope here. Software that uses the value for its documented purpose, deciding whether it can interpret the file, becomes *more* capable, not less.

- **Existing files:** No change. Files written by any prior CGNS version remain readable, and require no migration. Their `CGNSLibraryVersion_t` value is reinterpreted as a conservative minimum requirement, which yields the same accept/reject outcome the current library produces.
- **Existing applications (3-argument `cg_open`):** `cg_open` is not redefined, wrapped, or turned into a macro. It remains the same exported function with the same signature and the same behavior, and applications that use it inherit the global configuration exactly as they inherit `cg_configure` settings today. Source and binary compatibility are both unaffected, and no recompilation is required.
- **Default bounds** (`low = CG_LIBVER_AUTO, high = CG_LIBVER_DEFAULT`): Applications that do not call the new API functions inherit these defaults, which reproduce the current behavior—but with a feature-aware check instead of a version-number-only check. The sole behavioral change for existing code is that files from a *newer* library version that use no incompatible features will now be accepted instead of rejected.
- **Already-deployed libraries reading new files:** This is the property the redefinition exists to provide. An unmodified library of any prior version accepts a CPEX-0049-written file whenever the minimum required version recorded in `CGNSLibraryVersion_t` falls within its range, and silently ignores `CGNSWritingLibraryVersion_t`. No recompilation or vendor cooperation is required.
- **Provenance for existing workflows:** Applications that ask "which library wrote this file?" through `cg_version` are unaffected—the function retains its meaning, reading the provenance node when present and `CGNSLibraryVersion_t` otherwise (Sections 3.2.1 and 3.4.4). Tools that read the node directly rather than through the MLL do need updating, which is why Section 3.10 is in scope for this CPEX.

- **File format additions** (see Section 3.2): `CGNSLibraryVersion_t` is redefined in meaning but unchanged in structure; one new node (`CGNSWritingLibraryVersion_t`) is added at the root level. Old libraries silently ignore the new node.

# 5 Fortran Bindings

The following Fortran subroutines are provided in the `cgns` module (`src/cgns_f.F90`):

```fortran
subroutine cg_set_libver_bounds_f(fn, low, high, ier)
    integer, intent(in)  :: fn, low, high
    integer, intent(out) :: ier

! Query bounds and/or minimum version.  low, high and min_version are
! all OPTIONAL, mirroring the NULL-able C output pointers; omitting
! min_version skips the derivation, which is O(N) where one is needed.
subroutine cg_get_libver_bounds_f(fn, low, high, &
                                        min_version, ier)
    use iso_c_binding, only: C_NULL_PTR, C_LOC
    integer,          intent(in)                    :: fn
    integer,          intent(out)                   :: ier
    integer,          intent(out), optional         :: low, high
    integer,          intent(out), optional, target :: min_version

! Minimum required version only (named form of the O(N) query)
subroutine cg_get_min_version_f(fn, min_version, ier)
    integer, intent(in)  :: fn
    integer, intent(out) :: min_version, ier

! Typed setter on a parameter object
subroutine cg_params_set_libver_bounds_f(params, low, high, ier)
    type(C_PTR), intent(in)  :: params
    integer,     intent(in)  :: low, high
    integer,     intent(out) :: ier

! Parameter object open (4-argument form)
subroutine cg_open_with_params_f(filename, mode, params, fn, ier)
    use iso_c_binding, only: C_PTR
    character(*),     intent(in)  :: filename
    integer,          intent(in)  :: mode
    type(C_PTR),      intent(in)  :: params
    integer,          intent(out) :: fn, ier
```

`cg_open_f` keeps its three-argument form and its meaning; the four-argument form is a separate, explicitly named subroutine rather than an overload of it. This mirrors the C decision of Section 3.4.5: the reader of a call site can see which entry point is being used without knowing how many arguments the generic interface accepts. A generic interface would be sound in Fortran, where overloading is a language feature rather than a preprocessor trick, but naming the two forms differently in C and identically in Fortran would leave the bindings describing different APIs.

For parallel applications, `cgp_open_with_params_f` is provided with the same signature.

**Note on `min_version` cost control.** Fortran 2003 `OPTIONAL` dummy arguments provide the same NULL-pointer gating as the C API. The wrapper implementation checks `PRESENT(min_version)`: if absent, it passes `C_NULL_PTR` to the C function, skipping the derivation entirely; if present, it passes `C_LOC(min_version)`, triggering it—at $O(1)$ for a CPEX-0049 file open for reading, and $O(N)$ otherwise (Section 3.4.3). This mirrors the C semantics without requiring a separate entry point. Note that `C_LOC` requires its argument to have the `TARGET` (or `POINTER`) attribute, so the dummy argument is declared `TARGET` above; an implementation may equivalently pass `C_LOC` of a local `TARGET` temporary and copy the result out, which avoids imposing any requirement on the caller's actual argument.

The `CG_LIBVER_*` integer constants are exported from the `cgns` module and may be used directly in Fortran code.

# 6 Examples

## 6.1 Example 1: Default Behavior (No API Call)

An application compiled against CGNS 4.4 opens a file written by CGNS 5.0 that contains only structured zones, coordinates, and flow solutions—all features present since CGNS 2.x. Under the current library this fails; under this CPEX it succeeds (with an optional warning).

```c
/* No compatibility version call -- defaults apply */
int fn;
if (cg_open("output_from_v5.cgns", CG_MODE_READ, &fn)
        != CG_OK) {
    /* Under current library: fails with version error.
       Under this CPEX: succeeds because no v5-only
       features are present. */
    cg_error_print();
}
/* ... read data as usual ... */
cg_close(fn);
```

## 6.2 Example 2: Restricting to a Known-Compatible Range

A production solver sets the version bounds so that written files are compatible with a certified post-processing tool chain that only supports up to CGNS 4.0:

```c
/* Lock version bounds globally:
   low  = AUTO (write minimum needed),
   high = V40  (reject features beyond 4.0) */
cg_configure(CG_CONFIG_LIBVER_LOW,  (void *)(intptr_t)CG_LIBVER_AUTO);
cg_configure(CG_CONFIG_LIBVER_HIGH, (void *)(intptr_t)CG_LIBVER_V40);

int fn;
cg_open("solver_output.cgns", CG_MODE_WRITE, &fn);
/* AUTO write mode records the minimum version needed.
   high does not silently downgrade: a feature requiring
   more than 4000 makes the write call return CG_ERROR. */
/* ... write grid, solution, BCs ... */
cg_close(fn);
```

## 6.3 Example 3: Thread-Safe Per-File Configuration

Different threads open files with different version requirements:

```c
cg_parameters_t params;
cg_params_create(&params);

/* Floor at 4.0, no ceiling. Typed setter: no casts. */
cg_params_set_libver_bounds(params, CG_LIBVER_V40,
                                    CG_LIBVER_DEFAULT);

int fn;
cg_open_with_params("myfile.cgns", CG_MODE_WRITE,
                    params, &fn);
/* ... use file ... */
cg_close(fn);
cg_params_destroy(params);
```

## 6.4 Example 4: Analyzing Minimum Required Version

After writing a file, query the minimum version required by the features actually present:

```c
int fn, min_ver;
cg_open("output.cgns", CG_MODE_READ, &fn);
/* The named form: the O(N) scan is asked for explicitly */
cg_get_min_version(fn, &min_ver);
/* Two fractional digits: the encoding is dotted*1000,
   so 1050 must print as 1.05, not 1.5 */
printf("File requires CGNS >= %d.%02d\n",
       min_ver / 1000, (min_ver % 1000) / 10);
cg_close(fn);
```

## 6.5 Example 5: Using `cg_configure`

```c
#include <stdint.h>

/* Set bounds: AUTO lower, CGNS 4.0 upper */
cg_configure(CG_CONFIG_LIBVER_LOW,
             (void *)(intptr_t)CG_LIBVER_AUTO);
cg_configure(CG_CONFIG_LIBVER_HIGH,
             (void *)(intptr_t)CG_LIBVER_V40);

int fn;
cg_open("constrained.cgns", CG_MODE_WRITE, &fn);
/* ... */
cg_close(fn);
```

## 6.6 Example 6: Fortran Usage

```fortran
program libver_bounds_example
  use cgns
  implicit none
  integer :: ier, fn, lo, hi, min_ver

  ! Set global bounds: AUTO lower, CGNS 4.0 upper
  call cg_configure_f(CG_CONFIG_LIBVER_LOW,  CG_LIBVER_AUTO, ier)
  if (ier .ne. CG_OK) stop 'Error setting lower bound'
  call cg_configure_f(CG_CONFIG_LIBVER_HIGH, CG_LIBVER_V40,  ier)
  if (ier .ne. CG_OK) stop 'Error setting upper bound'

  call cg_open_f('test.cgns', CG_MODE_READ, fn, ier)
  if (ier .ne. CG_OK) then
     call cg_error_print_f()
     stop 'Open failed'
  end if

  call cg_get_libver_bounds_f(fn, lo, hi, min_ver, ier)
  write(*,*) 'Min version required:', min_ver

  call cg_close_f(fn, ier)
end program
```

## 6.7 Example 7: Detecting Incompatible Features

Three readers open the same kind of file—one written by CGNS 5.0 containing a `ParticleZone_t` node, which Section 3.7 registers at CGNS 4.5 (CPEX 0046). Under AUTO the file records `CGNSLibraryVersion = 4.50`. The three cases are worth stating together, because only the first is a diagnostic produced by this CPEX, and the third is what the proposal is measured against.

**(a) A CPEX-0049-aware reader older than the requirement.** A library at CGNS 4.6 reads `4.50` in $O(1)$, accepts, and reads the particle data. A library at CGNS 4.6 opening a file that requires `5.00`—one containing `ElementInterpolation_t`, say—is refused, and `_CGNS_FeatureMask` lets the message name the feature rather than only the number:

```c
int fn;
if (cg_open("interp_v5.cgns", CG_MODE_READ, &fn) != CG_OK) {
    /* "file requires CGNS >= 5.00 for ElementInterpolation_t" --
       the version alone decides; the mask supplies the name. */
    printf("%s\n", cg_get_error());
}
```

The verdict here is reached by the major-version guard of Step 2, before any of the file's contents are examined. Within a major version the same $O(1)$ comparison is made at Step 4. In neither case is a feature scan performed (Section 3.8).

**(b) The payoff case.** The same CGNS 5.0 writer, emitting only features present since CGNS 3.1, records `3.10`. Both the 4.6 library above and an unmodified, already-deployed CGNS 4.4 library accept it. Under the current library the 4.4 case fails, because the file would have declared `5.00`.

**(c) What a pre-CPEX-0049 reader actually does with case (a)'s file.** It is worth being exact, because it is not a rejection. An unmodified CGNS 4.4 library reading `4.50` compares major versions, finds them equal, issues `cgi_warning` ("the file being read is more recent that the CGNS library used"), and proceeds; the `ParticleZone_t` node, whose label it does not recognize, is silently ignored, as unknown labels always are. This is *pre-existing* behavior—a 4.4 library reading a 4.5 file does the same today—and this CPEX neither improves nor worsens it: old readers were never protected within a major version, and no change to the file can teach them to be. What the redefinition changes is the *across*-major case, which is a hard rejection today and becomes an acceptance whenever the file's real requirement permits it. Protection against silently-dropped content for readers that do want it is what `cg_get_min_version` (Section 3.4.3) and the `high` bound on the writing side are for.

# 7 Reference Implementation

A working implementation is available on the feature branch:

https://github.com/brtnfld/CGNS/tree/915

**Status against this revision.** The branch was written against version 1.1 and predates the design specified here; it demonstrates feasibility rather than conformance. The work items it implies are, in order:

1. Replace the separate `CGNSMinRequiredVersion_t` node with the redefined `CGNSLibraryVersion_t` plus `CGNSWritingLibraryVersion_t`.
2. Separate the acceptance value from the in-memory provenance value (Section 3.2.1), and remove the existing `CG_MODE_MODIFY` version rewrite in `cg_open`.
3. Replace write-path version accumulation with the close-time derivation of Section 3.6.1; the branch's incomplete version-bump map ceases to matter once the derivation is the source of truth.
4. Add the encoding-generation floors, then the completeness test that gates adoption (Section 3.6).
5. Treat an explicit `low` as a floor rather than a cap, and remove read-side enforcement of `high` (Section 3.5).
6. Restrict the feature scan to the legacy branch of the open path, key the legacy/CPEX-0049 distinction on `CGNSWritingLibraryVersion_t` alone rather than on the presence of the mask, and never refine a CPEX-0049 file's declared minimum downward (Sections 3.5 and 3.8).
7. Drop the polymorphic `cg_open` macro in favor of `cg_open_with_params`, and add the typed parameter setters.

Key changes in the reference implementation:

- `src/cgnslib.h` — Defines the `CG_LIBVER_*` integer constants, `CG_CONFIG_LIBVER_LOW` through `CG_CONFIG_GET_LIBVER_HIGH` tokens, `CG_PARAM_KEY` enum, `cg_parameters_t` type, and all new API prototypes.
- `src/cgnslib.c` — Implements `cg_set_libver_bounds`, `cg_get_libver_bounds`, `cg_params_create/destroy/set`, `cg_open_with_params`, and the revised open/write logic.
- `src/cgns_internals.c` — Feature-version registry with detector functions; `cgi_require_version` for AUTO write-version tracking; `cgi_check_version_limit` for fast-path validation.
- `src/cgns_f.F90` — Fortran module wrappers for all new functions and the overloaded `cg_open` interface.
- `src/tests/test_version_bounds.c` — Twelve test cases covering AUTO write mode, explicit version locking, compatibility enforcement on read, compatibility inheritance by MODIFY mode, feature-mismatch error reporting, per-file compatibility versions, and legacy-file healing. The CGNS 5.0 high-order element case is presently a stub that reports `SKIP`; the completeness test mandated in

Section 3.6 supersedes it and must be implemented before adoption.

- `src/tests/test_parameters.c` — Seven test cases for the parameter object lifecycle and `cg_open_with_params`. The cases covering the withdrawn polymorphic macro are removed.

# 8 Edge Cases and Limitations

## 8.1 External Links

CGNS supports linked nodes that reference data in external files. Compatibility checking for external links works correctly without user intervention, but the underlying mechanism exposes a pre-existing architectural concern that deserves acknowledgment.

**Current behavior.** Both the HDF5 (ADFH) and ADF backends resolve external links transparently at the I/O layer. More critically, `cgi_read()` — which is called *before* the feature scan during `cg_open()` — reads the entire file tree eagerly, following all external links and opening all linked files at startup. The feature scan therefore walks already-populated structs; it adds no additional file opens beyond what `cgi_read()` already performed. External link content is fully visible to all detectors.

**The HPC I/O storm concern.** On parallel file systems such as Lustre or GPFS, opening and parsing dozens of external linked files during `cg_open()` of a master file can cause severe metadata latency—an "I/O storm." This is a pre-existing concern with CGNS's eager `cgi_read()` architecture and is not introduced by this CPEX. However, the addition of compatibility checking makes it more important to resolve, since applications may now call `cg_open()` with restrictive compatibility versions specifically to validate linked content, amplifying the existing performance problem.

**Three architectural strategies for future resolution.** The following strategies address both the I/O storm and version checking for external links. They are presented as candidates for a follow-on CPEX or as input to an architectural review; they are not required for the initial implementation of CPEX 0049.

**Strategy 1: Deferred just-in-time validation (recommended).** Make `cgi_read()` lazy: defer reading a linked node until the application first traverses into it (e.g., via `cg_goto`). The compatibility check for that node is performed at the same moment, deep in the traversal path rather than at startup. This eliminates the I/O storm entirely and gives the application a `CG_ERROR` at the exact point of incompatibility. The trade-off is that error detection moves from `cg_open()` to the first traversal call, requiring consistent `CG_ERROR` checking throughout read loops. This strategy requires significant refactoring of the CGNS read path and is a larger effort than CPEX 0049 alone.

**Strategy 2: Embedded link version metadata.** When `cg_link_write()` creates a link to an external file, the library briefly opens the target, computes its minimum required version, and stores the result as a hidden attribute (e.g., `_CGNS_LinkMinVersion`) on the link node in the parent file. Subsequent `cg_open()` calls can read this $O(1)$ attribute without opening the external file. The limitation is staleness: if the external file is modified after the link is created, the cached attribute becomes incorrect. This strategy requires a file-format change (new hidden attribute) and a corresponding SIDS/file mapping update.

**Strategy 3: Write-time cascading version inheritance.** When `cg_link_write()` creates a link, the library opens the target file and obtains its minimum required version—$O(1)$ from the recorded value for a CPEX-0049 target, or by derivation for a legacy one—and retains it with the link node in the in-memory tree. The close-time derivation of Section 3.6.1 then takes the maximum of the parent's own content-derived minimum and the requirement of every link it contains, so the parent's recorded minimum covers the union of both files' requirements.

*This must be specified as an input to the close-time derivation, not as a `cgi_require_version()` call.* Earlier revisions described the cascade in those terms, when an accumulated version was the source of truth; under Section 3.6.1 that call no longer determines the recorded value, and a cascade routed through it would never reach the file. Two consequences follow, both benign. Because link creation is a collective metadata call, every rank obtains the same target requirement and the parallel argument of Section 3.6 is unaffected. And because the parent's `_CGNS_FeatureMask` describes the parent's own content, a cascaded requirement raises the recorded minimum above what the parent's mask implies—the same situation an explicit `low` produces, and not a disagreement in the sense that matters (Section 3.8.4).

A downstream reader opening the parent file will then see a version header that reflects the highest-version feature reachable via any link, and can reject the file immediately based on that header alone without opening any linked file. This strategy is the most compatible with the current CPEX 0049 design and requires no structural changes to `cg_open()`. It is also the strategy that makes an existing provision of the file mapping true rather than merely hoped for: the mapping already states that "it is assumed that the version number in the `CGNSLibraryVersion_t` node in the 'top-level' file is applicable to the file(s) containing the linked nodes." Cascading each link target's requirement into the parent's recorded minimum is precisely what discharges that assumption.

**Which strategy is best?** The answer depends on which problem is being solved. A precise comparison requires distinguishing two cases: a *reject* (the version check fails and the file is refused) and an *accept* (the check passes and the file is opened normally).

| | **Strategy 1** | **Strategy 2** | **Strategy 3** |
|---|---|---|---|
| Eliminates I/O storm on reject | ✓ | ✓ | ✓ |
| Eliminates I/O storm on accept | ✓ | × | × |
| Requires file format change | × | ✓ | × |
| Staleness risk | × | ✓ | ✓ |
| Scope compatible with CPEX 0049 | × | × | ✓ |

A critical observation: Strategies 2 and 3 only improve the *reject* path. In the *accept* path, `cgi_read()` still runs and still opens every linked file regardless of which strategy is in place. The I/O storm in the accept case is eliminated *only* by Strategy 1 (lazy `cgi_read()`).

Strategy 2 is strictly inferior to Strategy 3 within the scope of this CPEX: it requires a file-format change and a SIDS update, carries the same staleness risk, provides the same fast-reject benefit, and adds no advantage in the accept case.

Strategy 1 is architecturally the correct long-term solution—it is the only strategy that eliminates the I/O storm in both the accept and reject paths—but it requires refactoring the entire CGNS read path into a lazy model, which is a substantially larger effort than this CPEX and warrants its own proposal.

**Recommendation:** Implement Strategy 3 in this CPEX as the lowest-risk, highest-compatibility improvement. File a separate architectural proposal for Strategy 1 (lazy `cgi_read()`) as a follow-on effort that would benefit all CGNS users on HPC file systems, not only those using compatibility version checking.

**Out-of-band update risk (Strategy 3).** Strategy 3 is vulnerable to external file modifications that occur *after* the link is established. If File A links to File B when both require CGNS 3.0, and File B is later modified out-of-band to include a `ParticleZone_t` node (requiring CGNS 4.5), File A's version header is not updated automatically. A CGNS 3.0 reader opening File A will pass the header check and then encounter an incompatible node upon traversing the link, producing a delayed error instead of a clean rejection at open time. This limitation is inherent to any write-time caching approach (including Strategy 2) and must be documented clearly in the MLL reference manual. Users who modify external files after linking should reopen the parent file in `CG_MODE_MODIFY` and close it immediately; this recomputes the version header and `_CGNS_FeatureMask` attribute. This workaround is straightforward: only `CGNSLibraryVersion_t` is recomputed, while `CGNSWritingLibraryVersion_t` records the library that performed the recomputation—provenance is preserved throughout.

## 8.2 Version Downgrades on Node Deletion

Deleting the last node that required a high version—removing the only `ParticleZone_t` from a file, say—must lower the recorded minimum, or the file permanently over-states its requirement and an older reader that could now read it is told it cannot.

This case needs no special handling under the derivation rule of Section 3.6.1, and that is one of the reasons the rule was adopted. Because the minimum is computed at close from the features actually present in the tree, and not accumulated from the history of what was written, a deleted feature simply stops contributing. The recorded value falls to whatever the remaining content requires, in the same traversal that would have raised it had a feature been added instead. Downgrade is not a special path; it is the absence of one.

An incremental design would have required the opposite: a decremental bookkeeping mechanism in which every deletion path identifies which feature bit it may clear, plus a reverse lookup from feature to bit, plus a rule for the case where two nodes contribute the same bit and only one is deleted. That machinery would have to be re-audited every time a delete path is added. Deriving from content makes the entire question disappear rather than answering it.

**Interaction with the feature mask.** The `_CGNS_FeatureMask` attribute is written from the same derivation, so it cannot drift out of agreement with the tree it describes—including after deletions—because it is never carried forward from the previous value. The two exceptions are stated where they arise: a crash between the modification and `cg_close()` leaves both values describing the file as it was (Section 3.8.4), and a raised floor or a cascaded link requirement lifts the recorded minimum above what the mask implies by design. Neither can turn a rejection into an acceptance, since no reader consults the mask for a verdict.

## 8.3 Partial Parallel Writes and Version Agreement

In an MPI-IO scenario where different ranks write different data concurrently, the minimum required version recorded in the file must cover every feature present in the file.

**AUTO mode.** No coordination is required, at either the application level or the library level. Because metadata-defining calls in parallel CGNS are collective with identical arguments, the set of *features* present is replicated on every rank even when the *data* written is not; deriving the minimum at close from that replicated metadata therefore produces the same value everywhere (Section 3.6.1). Ranks cannot disagree, so nothing has to be reconciled.

**Explicit `low` bound.** When an explicit lower bound is configured, all ranks must set the same value before calling `cgp_open`. If any rank attempts to write a feature whose minimum version exceeds the configured `high` bound, the library returns `CG_ERROR` on that rank. This error is rank-local; other ranks are not automatically notified. Applications must treat a per-rank `CG_ERROR` from a write call as a collective failure requiring abort or coordinated recovery.

# 9 Design Decisions and Ratification

This proposal specifies a single design. Where a choice existed, it has been made and argued in the section that defines the affected behavior; this section is an index to those decisions, so that a reviewer can see what was considered without hunting for it, and so that settled matters are not reopened by default.

Version 1.3 closes the questions that version 1.2 left to the committee. Each was resolved on evidence—the behavior of the precedent this CPEX cites, the contents of the CGNS sources, or a property that can be checked—rather than on preference, and the evidence is given at the point of decision. What remains for the committee is the part that is properly theirs: whether to amend the standard.

## What the Steering Committee is being asked to approve

**Adopted 2026-08-04.** The Steering Committee voted to reverse its 2026-04-14 preference for a purely additive design and accepted the redefinition below, on the case made in Section 2.2: an additive node is invisible to exactly the readers whose behavior must change, so it delivers no benefit to any library already in the field. All three items were adopted for SIDS, Filemap and MLL; the reference implementation predates this design and its remaining work items are tracked in Section 7.

Three items required a vote, because each changes the standard rather than the implementation:

1. **The redefinition of** `CGNSLibraryVersion_t` from “the version of the CGNS standard with which the file is consistent” to “the minimum version of the CGNS standard required to read the file,” amending both the SIDS (*Hierarchical Structures*, “CGNS Version”) and the file mapping node description (Section 3.2). This is the substance of the proposal; everything else follows from it.
2. **The addition of** `CGNSWritingLibraryVersion_t` as an officially recognized root-level node, with the node description given in Section 3.11. The file mapping already permits unrecognized root children, so this makes the node official rather than legal.
3. **The scope commitment** that the tool updates of Section 3.10 ship with the change rather than after it. Without them the distribution would report

provenance incorrectly for files written by its own library.

Everything below is an implementation or API decision within the champion's remit, recorded for review rather than for a vote. Any of them can of course be revisited if a reviewer finds the reasoning wrong—but each is stated as a decision, with its reason, rather than as a question.

## Decisions taken in this revision

**File format: redefine rather than add.**
`CGNSLibraryVersion_t` carries the minimum required version; provenance moves to the new node. Version 1.1 had adopted the purely additive alternative; it is reversed here on the grounds of Section 2.2—an additive change reaches no already-deployed reader, and reaching them is the proposal's entire purpose.

**The recorded minimum is derived at close, not accumulated.**
Running the registry's detectors over the in-memory tree at `cg_close` is normative (Section 3.6.1). The incremental alternative makes every file's correctness depend on a maintainer remembering to add a call, and fails silently when one is missed. `cgi_require_version()` is retained only to report `high` violations at the offending call rather than at close, so that the fragile mechanism owns the benign failure mode.

**The AUTO floor accounts for encoding generation, not just features.**
Normative, and the correctness of the whole scheme depends on it: a minimum below an encoding boundary makes an older reader *mis-decode* modern bytes rather than reject them (Section 3.6).

**`low` and `high` govern writing only.**
Neither takes part in the acceptance decision (Section 3.5). HDF5 specifies the same for `H5Pset_libver_bounds`, the party a ceiling protects is in another process and never sees the setting, and read-side enforcement would make an application refuse files it is capable of reading. This also disposes of the read-only-leniency question raised in version 1.2: with no read-side gate there is nothing to relax.

**The recorded minimum is normative on read; the mask never overrides it.**
A CPEX-0049 file's declared requirement is honored as given and is not refined downward against the file's contents (Section 3.5). Refinement would require reconstructing the derived minimum, and `_CGNS_FeatureMask` cannot support it—the encoding-generation floors are not features and carry no bits—while overriding the declaration would defeat the explicit floor that Section 3.7 prescribes as the only remedy for out-of-band writes. The refinement in Step 4 therefore applies to legacy files alone, whose declared value is provably not a requirement. Recording the derived minimum as a second on-disk value was considered and rejected there.

**The open path performs no feature scan for any conforming file.**
Stated as an invariant rather than a performance hope: CPEX-0049 files are never

scanned, and by the adoption invariant of Section 3.5 a legacy file's declared version is always below that of a CPEX-0049-aware reader, so the $O(N)$ fallback is reachable only for a non-conforming writer or an externally edited version node. The traversal survives as the close-time derivation and as `cg_get_min_version`, not as a step in `cg_open`.

**An explicit `low` is a floor, not a cap.**
A feature requiring more than `low` raises the recorded version rather than failing; only `high` produces an error (Section 3.6).

**`high` defaults to `CG_LIBVER_DEFAULT`, not `CG_LIBVER_LATEST`.**
`CG_LIBVER_LATEST` is a compile-time constant, so defaulting to it would let an application silently forbid a newer library from writing newer features after a link-time upgrade—in an extension about mixed-version deployment. `CG_LIBVER_DEFAULT = 0` is not a version number and cannot go stale (Section 3.3). HDF5's `H5F_LIBVER_LATEST` has this defect; CGNS need not inherit it.

**The new node is `R4`, with a normative rounding rule.**
$\lfloor v \times 1000 + 0.5 \rfloor$ makes the binary32 round trip exact for every representable CGNS version, the distribution already relies on that rule in `cg_version` and `cgnscheck`, and `R4` keeps one decode path and one file-mapping description for both nodes (Section 3.2).

**The node is named `CGNSWritingLibraryVersion_t`.**
"Creator" is factually wrong, "writer" is not a SIDS term of art, and the sibling's name is no longer self-describing—so the companion must carry the disambiguation (Section 3.2). Placement as a root sibling rather than a child or attribute is argued in Section 3.2.2.

**`_CGNS_FeatureMask` is diagnostic, not normative.**
An unrecognized bit in an otherwise-acceptable file means "a feature I do not know is present, and the writer says it does not require a newer library"—which describes a forward-compatible feature, not a hazard. Rejecting on it would refuse the files this CPEX exists to admit. The recorded minimum is the general fail-safe (Section 3.8.1).

**`cg_open` remains an ordinary function.**
The polymorphic variadic-macro form is withdrawn: `__VA_ARGS__` appears nowhere in CGNS today, argument counting is unreliable on vendor preprocessors, and a macro cannot be resolved through `dlsym` or an FFI. `cg_open_with_params` is the sole four-argument entry point and `CGNS_NO_MACROS` is unnecessary (Section 3.4.5).

**Parameter objects use typed setters.**
`cg_params_set_libver_bounds` and its siblings are the documented interface, mirroring `H5Pset_libver_bounds`; the generic `void *` setter is retained only as an extension point (Section 3.4.5).

**`cg_get_min_version` is provided.**
The one query that can cost more than $O(1)$ gets a name of its own, so that the cost is chosen rather than stumbled into (Section 3.4.3).

**Node deletion downgrades the recorded minimum.**
Automatically, as a consequence of deriving from content (Section 8.2). No decremental bookkeeping is specified because none is needed.

**No `MPI_Allreduce` is added to `cgp_close`.**
Collective metadata calls leave every rank with identical input to the derivation, so ranks cannot disagree (Section 3.6).

**External links use Strategy 3 (write-time cascading).**
It is the only one of the three candidates that fits this CPEX's scope, and it discharges an assumption the file mapping already makes about linked files (Section 3.10 and the external links discussion). The cascade is specified as an input to the close-time derivation rather than as a `cgi_require_version()` call; routed through the latter, as earlier revisions had it, it would never reach the file. Lazy `cgi_read()` remains the correct long-term answer to the I/O storm and belongs in its own proposal.

**Tool updates are in scope.**
Section 3.10.

## 10 Summary of API Additions

| Symbol | Description |
|---|---|
| `CG_LIBVER_EARLIEST` | Oldest CGNS library version (1050). |
| `CG_LIBVER_LATEST` | Current library version (5000). |
| `CG_LIBVER_AUTO` | Sentinel ($-1$): automatic write-version selection. |
| `CG_LIBVER_DEFAULT` | Sentinel (0): no write ceiling; tracks the library linked at run time. Default for `high`. |
| `CG_LIBVER_V12`...`CG_LIBVER_V50` | Specific version milestone constants: `V12`, `V30`, `V31`, `V32`, `V40`, `V45`, `V50` (1200, 3000, 3100, 3200, 4000, 4500, 5000). |
| `CG_CONFIG_LIBVER_LOW` | `cg_configure` token: set global lower bound. |
| `CG_CONFIG_LIBVER_HIGH` | `cg_configure` token: set global upper bound. |
| `CG_CONFIG_GET_LIBVER_LOW` | `cg_configure` token: query global lower bound. |
| `CG_CONFIG_GET_LIBVER_HIGH` | `cg_configure` token: query global upper bound. |

| Symbol | Description |
|---|---|
| CGNS_FEATURE_* | Power-of-two mask constants for _CGNS_FeatureMask (bits 0–9; next free bit 10). |
| CG_PARAM_KEY | Enum of parameter keys (100–103). |
| cg_parameters_t | Opaque parameter object type. |
| CG_PARAMS_DEFAULT | NULL sentinel: use global configuration. |
| cg_set_libver_bounds | Set per-file version bounds (fn, low, high). |
| cg_get_libver_bounds | Query per-file bounds and/or minimum required version. Any output pointer may be NULL; the derivation runs only when min_version is non-NULL, and costs $O(N)$ only for a legacy file or a file open for writing. |
| cg_params_create | Allocate parameter object with defaults. |
| cg_params_destroy | Free parameter object. |
| cg_params_set | Set a parameter by key/value. |
| cg_open_with_params | Open file with explicit parameter object (standard name). |
| cgp_open_with_params | Parallel open with explicit parameter object. |
| cg_get_min_version | Query only the minimum version the file's content requires; the named form of the one query that is not always $O(1)$. |
| cg_params_set_libver_bounds | Typed setter for the version bounds on a parameter object (with _file_type and _compress siblings). |
| CGNSLibraryVersion_t | **Redefined**: now records the minimum CGNS version required to read the file, rather than the writing library's version. |
| CGNSWritingLibraryVersion_t | New SIDS root node recording the library version that last wrote the file (provenance). |
| _CGNS_FeatureMask | Hidden 64-bit signed attribute (SIDS I8; HDF5: H5T_NATIVE_INT64): bitmask of features present; enables precise diagnostics and unknown-feature detection. Placed on CGNSLibraryVersion_t. Requires a new cgio attribute API; no ADF realization (Section 3.8). |
| cg_set_libver_bounds_f | Fortran: set per-file version bounds (fn, low, high). |

| Symbol | Description |
|---|---|
| `cg_get_libver_bounds_f` | Fortran: query per-file bounds and (optionally) minimum required version. `min_version` is an `OPTIONAL` dummy argument (F2003); when absent, `C_NULL_PTR` is passed to the C layer, skipping the $O(N)$ scan. |
| `cg_open_with_params_f` | Fortran: 4-argument open with params. |
| `cgp_open_with_params_f` | Fortran (parallel): 4-argument open with params. |
| `cg_get_min_version_f` | Fortran: query the minimum required version. |

## 11 References


1. CGNS GitHub Issue #915, "Add CGNS version bounds control,"
   https://github.com/CGNS/CGNS/issues/915
2. CGNS GitHub Issue #670, "Buffer overflow in `cgi_error` using v3.31 thru v4.2" (related version-check code),
   https://github.com/CGNS/CGNS/issues/670
3. CGNS Confluence, "Resolve issue with release's 3.4.0 version compatibility, the 4.0.0 release, and forward compatibility,"
   https://cgnsorg.atlassian.net/wiki/spaces/CGNS/pages/220463122/
4. HDF5 Reference Manual, `H5Pset_libver_bounds`,
   https://docs.hdfgroup.org/hdf5/v1_14/group___F_A_P_L.html
5. CGNS Standard Interface Data Structures (SIDS),
   https://cgns.org/standard/SIDS/CGNS_SIDS.html
6. CGNS Mid-Level Library—File Operations,
   https://cgns.org/standard/MLL/api/c_api.html